Efficient Memory Management for Large Language Model Serving

이진호[1], 배진우[1], 이상백[1]

[1] : Kookmin University, Computer Science

Intro

Intro: AI - Large Language Model



Chat GPT

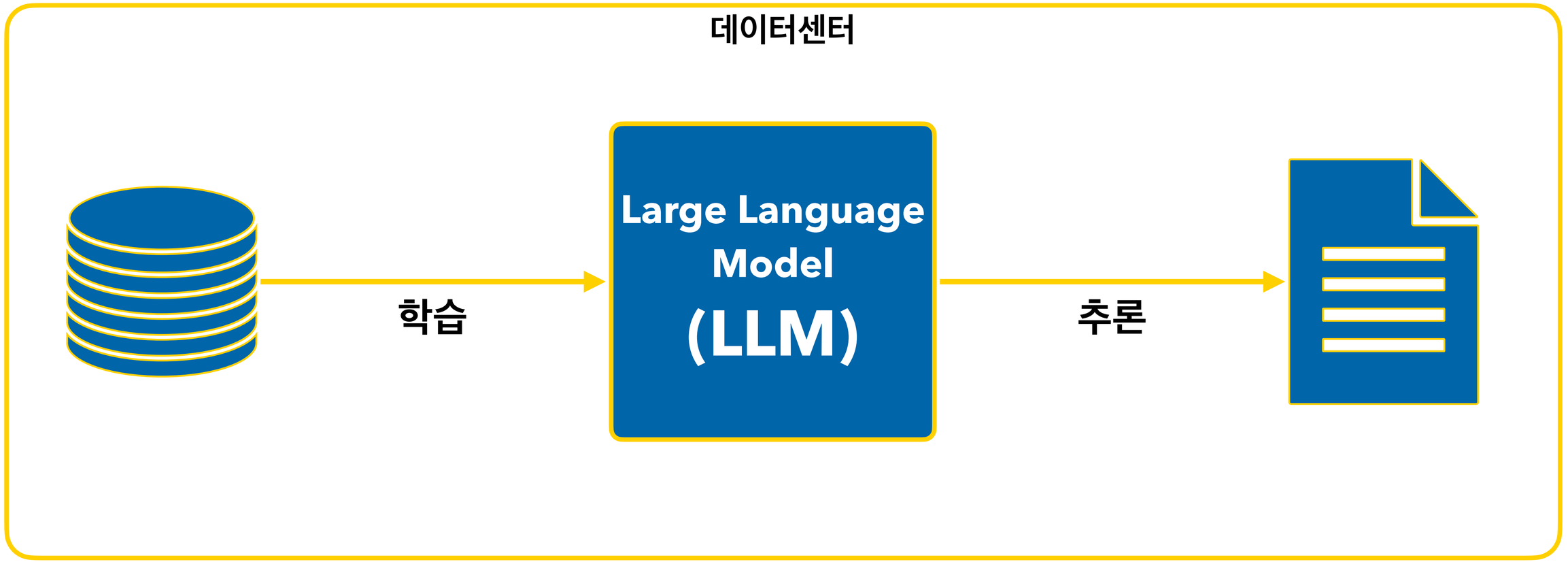


Gemini

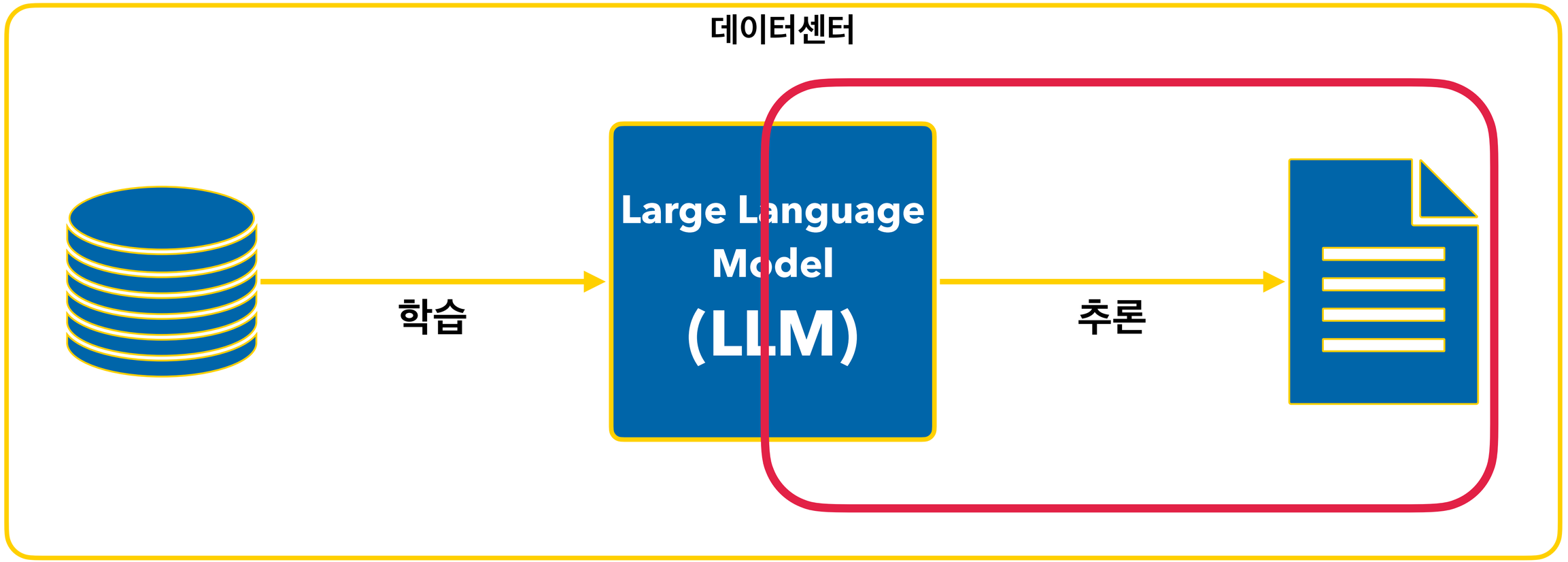


Github copilot

Intro: Training & Inference



Intro: Training & Inference



Intro: Large Language Model – Transformer

Ashish Vaswani, et al. “Attention Is All You Need.”, (2017)

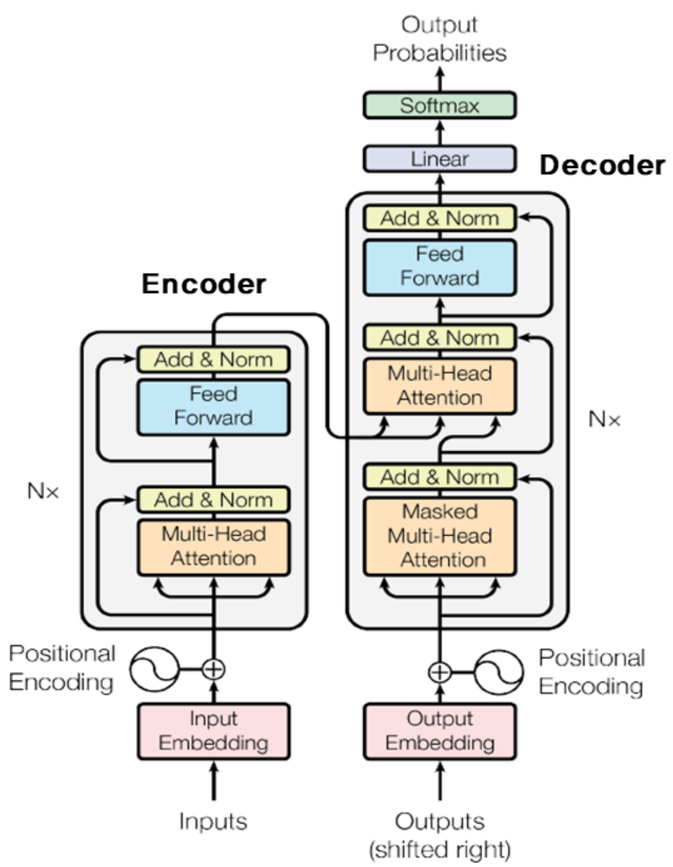
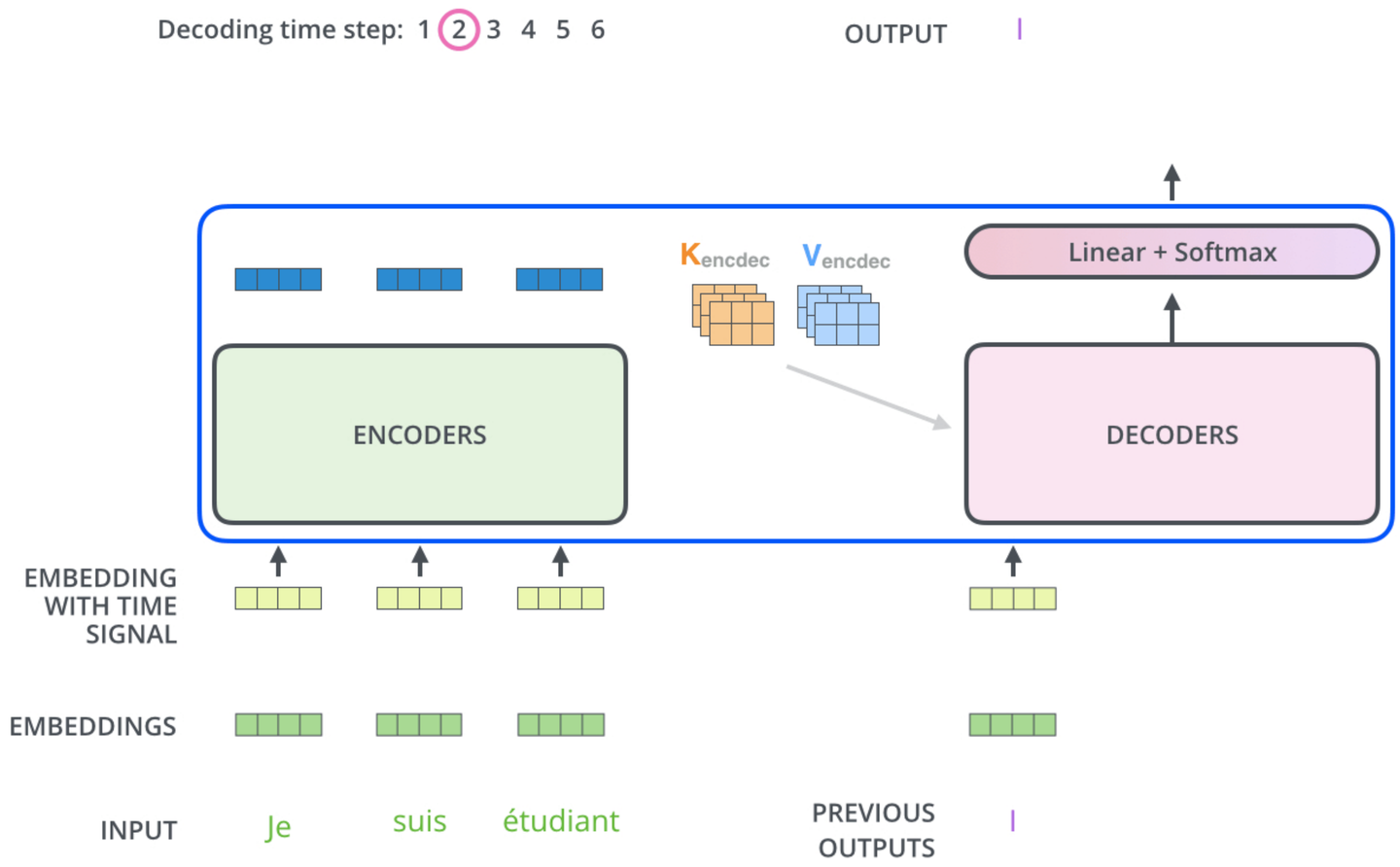


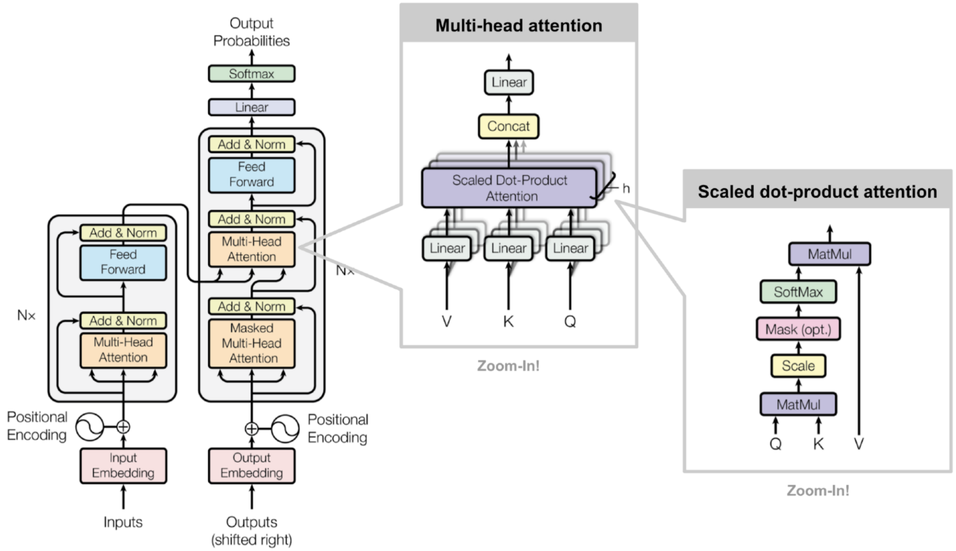
Figure 1: The Transformer - model architecture.

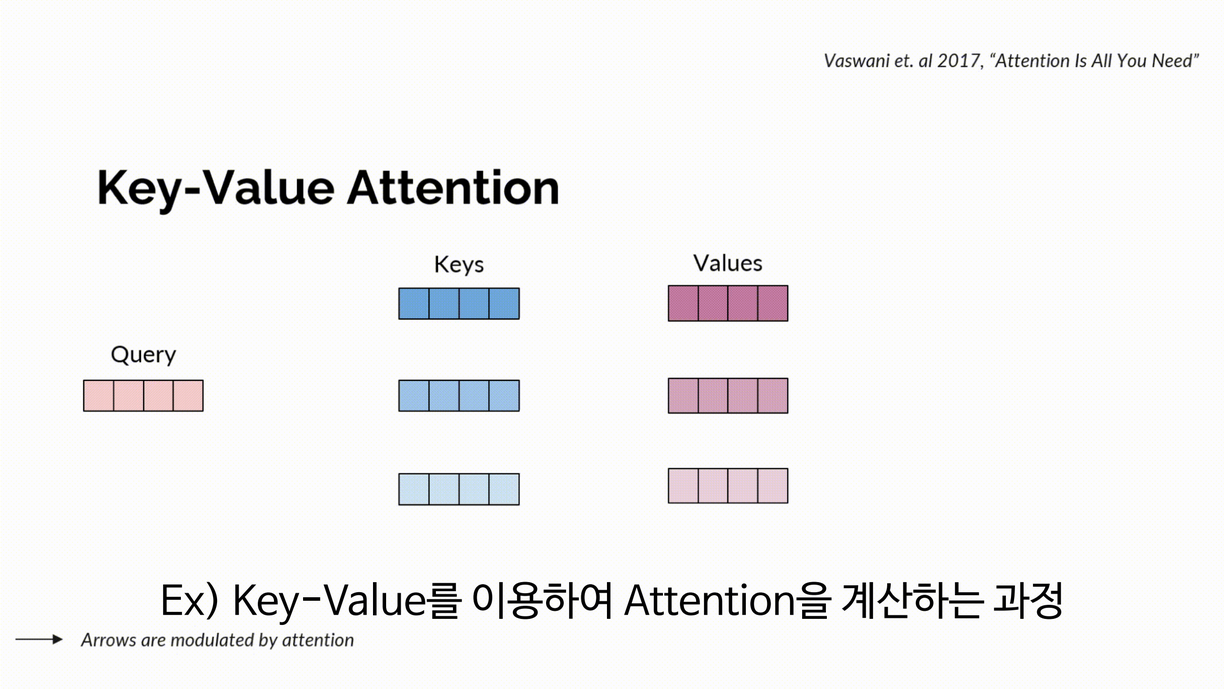
Transformer



간단한 Transformer 작동 과정

Intro: Large Language Model - Attention, KV-Cache





Ex) Key-Value를 이용하여 Attention을 계산하는 과정

Attention Mechanisms

- **Self-Attention**
- **Cross-Attention**
- **Multi-Head Attention**
- **Masked Multi-Head Attention**
- **Dot-Product Attention**
- **Scaled Dot-Product Attention**
- **Additive Attention**
- **…**

*KV cache(MHA): 2(K&V)*batch_size*n_layers*n_heads*d_head*seq_len*precision*

Multi Head Attention 기법을 이용할 때 KV-Cache가 차지하는 메모리 크기 계산

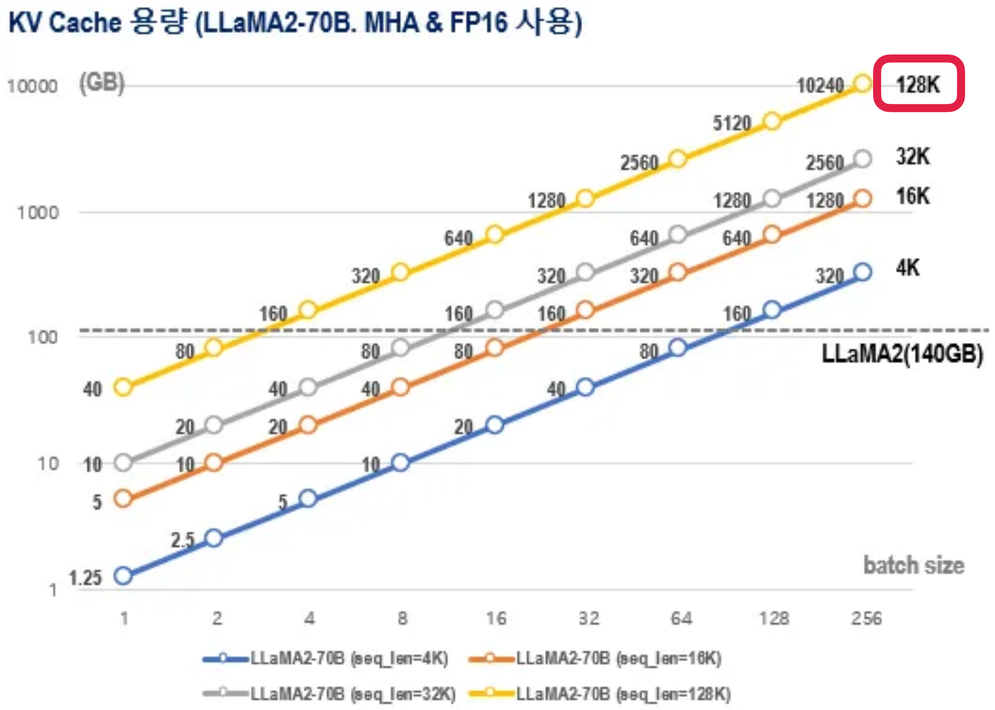
Alpha = 2(K&V) x Precision x Layers x …

KV-Cache 크기 = **Batch_size** x **Sequence_length** x Alpha

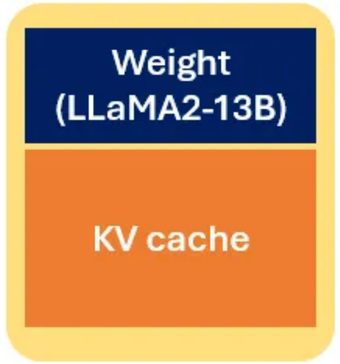
Problem

Problem: KV-Cache size

GPT-4o
context window size = **128K**



Sequenc_length와 BatchSize 에 따른 KV-Cache 메모리 크기 추이
(LLaMA2-70B 사용)



26GB (LLaMA2-13B. FP16)

54GB = 0.82MB*batch_size*seq_len
(LLaMA2-13B. FP16)

Memory capacity of A100 GPU :
Weight + KV cache = 80GB

KV-Cache 사용 가능 메모리 크기 = GPU 메모리 크기 - 모델 크기

How

How: vLLM, InfiniGen

Efficient Memory Management for Large Language Model Serving with PagedAttention
2023.09.12

vLLM

InfiniGen: Efficient Generative Inference of Large Language Models with Dynamic KV Cache Management
2024.06.28

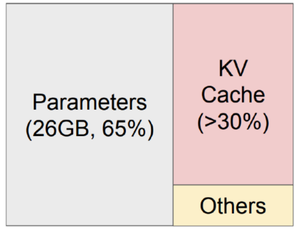
InfiniGen

vLLM

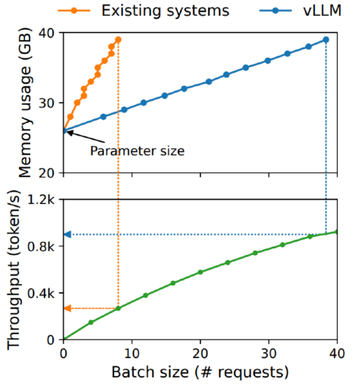
Efficient Memory Management for
Large Language Model Serving with PagedAttention
2023.09.12

How: vLLM – Problem

the LLM. Since the model weights are constant and the activations only occupy a small fraction of the GPU memory, the way the KV cache is managed is critical in determining the maximum batch size. When managed inefficiently, the KV cache memory can significantly limit the batch size and consequently the throughput of the LLM, as illustrated in Fig. 1 (right).



NVIDIA A100 40GB



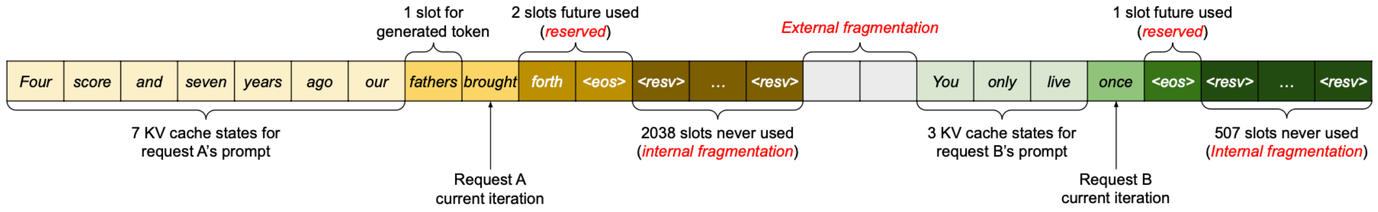
KV cache 잘못 관리하면 Batch 처리를 많이 못하고 성능 저하로 이어지는데?

ous memory. However, unlike the tensors in the traditional deep learning workloads, the KV cache has unique characteristics: it dynamically grows and shrinks over time as the model generates new tokens, and its lifetime and length are not known a priori. These characteristics make the existing systems' approach significantly inefficient in two ways:

KV Cache의 생명주기, 중요도, 얼마나 증가하고 줄어들지도 모르는 특징이 있는데…
기존 시스템은 KV cache 잘못 관리한 2가지 특징이 있다…

First, the existing systems [31, 60] suffer from internal and external memory fragmentation. To store the KV cache of a request in contiguous space, they *pre-allocate* a contiguous chunk of memory with the request's maximum length (e.g., 2048 tokens). This can result in severe internal fragmentation, since the request's actual length can be much shorter than its maximum length (e.g., Fig. 11). Moreover,

첫 번째는 내부 단편화, 외부 단편화가 너무 심하다….



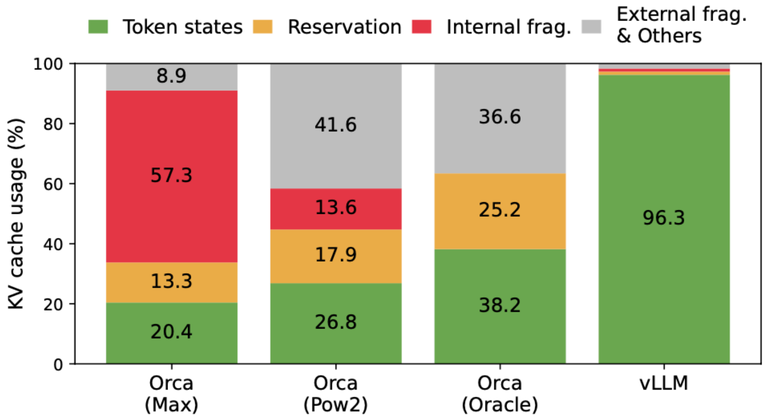
How: vLLM – Problem

Second, the existing systems cannot exploit the opportunities for memory sharing. LLM services often use advanced decoding algorithms, such as parallel sampling and beam search, that generate multiple outputs per request. In these scenarios, the request consists of multiple sequences that can partially share their KV cache. However, memory sharing is not possible in the existing systems because the KV cache of the sequences is stored in separate contiguous spaces.

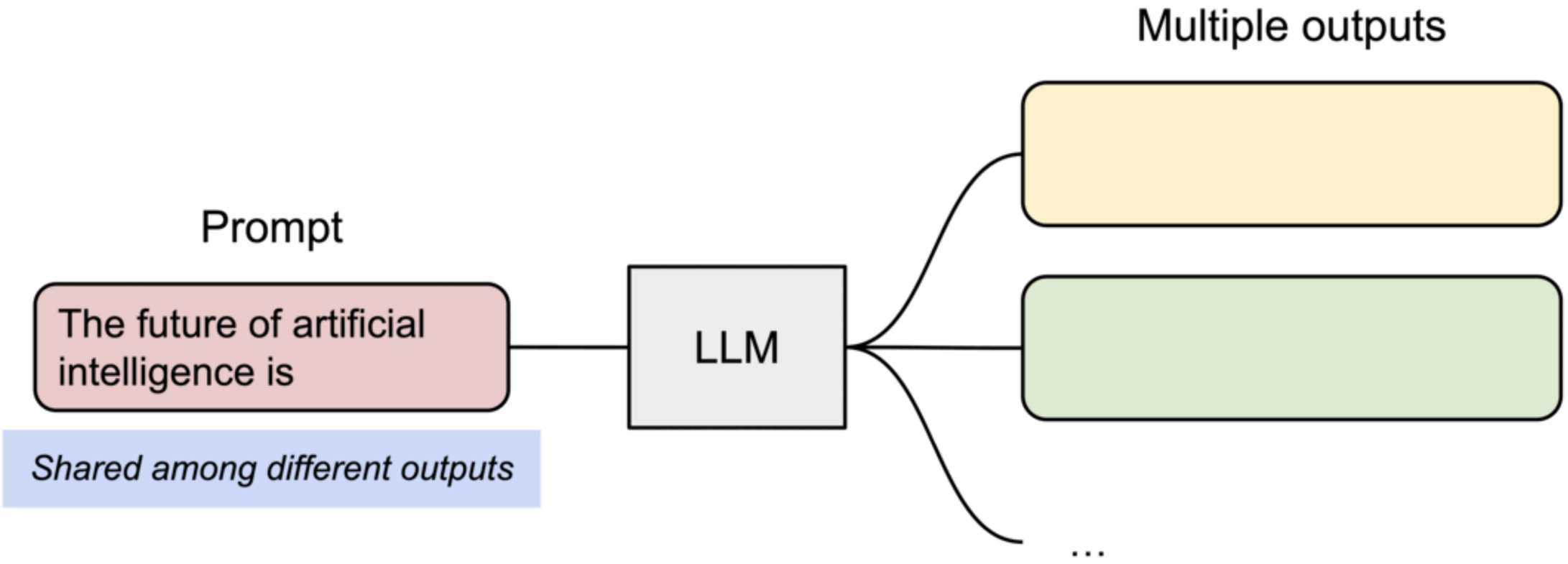
두 번째는 Beam search, Parallel Sampling 을 할 때 중복되는 KV Cache 를 공유하지 못한다.
왜? 연속적인 메모리 공간을 가지기 때문에…

To address the above limitations, we propose *PagedAttention*, an attention algorithm inspired by the operating system's (OS) solution to memory fragmentation and sharing: *virtual memory with paging*. PagedAttention divides the request's KV cache into blocks, each of which can contain the attention keys and values of a fixed number of tokens. In PagedAttention, the blocks for the KV cache are not necessarily stored in contiguous space. Therefore, we can manage the KV cache in a more flexible way as in OS's virtual memory: one can think of blocks as pages, tokens as bytes, and requests as processes. This design alleviates internal fragmentation by using relatively small blocks and allocating them on demand. Moreover, it eliminates external fragmentation as all blocks have the same size. Finally, it enables memory sharing at the granularity of a block, across the different sequences associated with the same request or even across the different requests.

그래서 우리는 이 한계를 해결하기 위해 OS 알고리즘에 영감을 받아 만든 PagedAttention을 제안한다.
이것은 메모리 단편화를 줄이고 KV Cache 공유를 가능하게 한다.

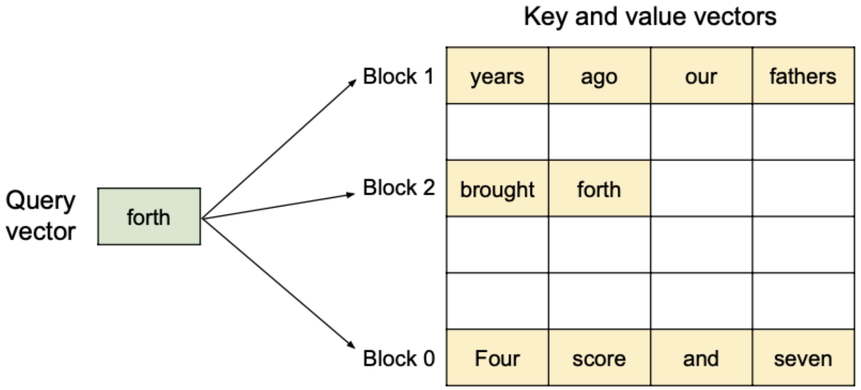


Beam Search, Parallel Sampling?



How: vLLM – Method: PagedAttention

PagedAttention



Key Idea: 블록 단위 Attention 연산

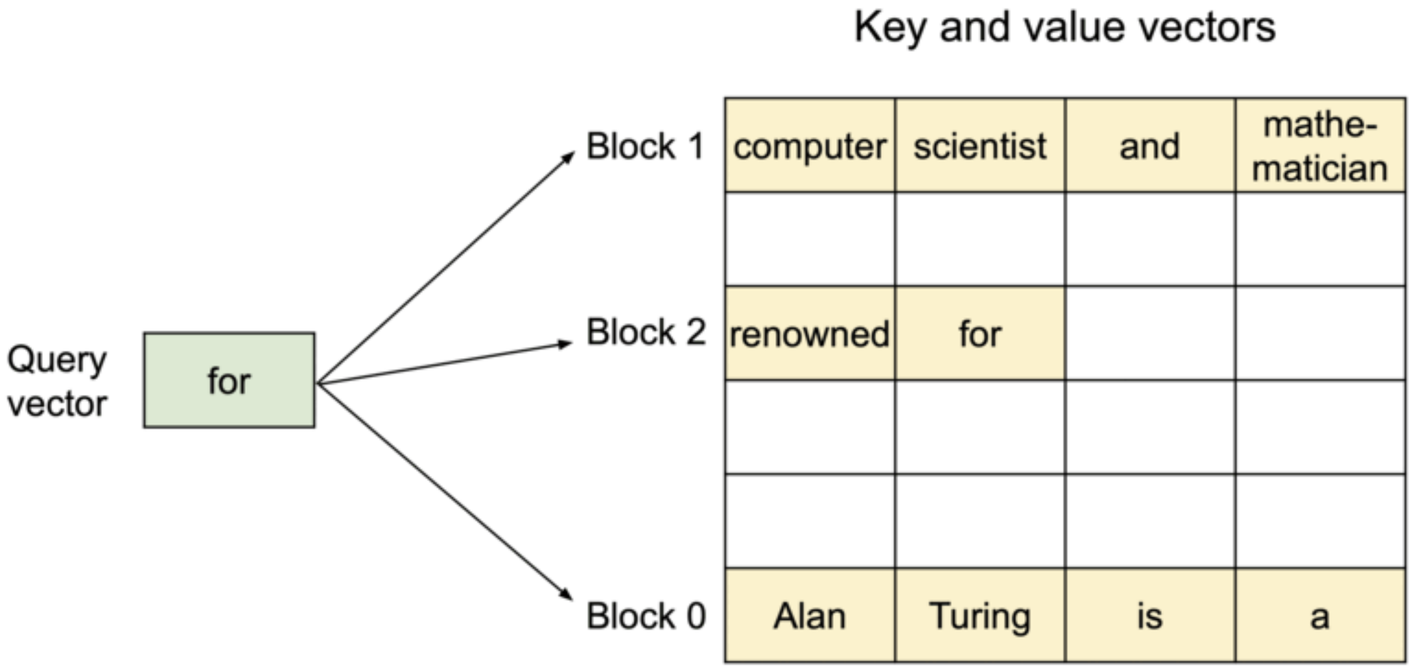
비연속적인 메모리 저장과 페이지 단위로 Attention을 계산하고 모두 합하여 전체 Attention을 계산 할 수 있다.

$$A_{ij} = \frac{\exp(q_i^\top K_j / \sqrt{d})}{\sum_{t=1}^{\lceil i/B \rceil} \exp(q_i^\top K_t \mathbf{1} / \sqrt{d})}, \quad o_i = \sum_{j=1}^{\lceil i/B \rceil} V_j A_{ij}^\top, \qquad (4)$$

- q_i: 현재 Query 벡터입니다. 현재 계산 중인 Query 토큰과 관련된 벡터입니다.
- K_j: j-번째 Key 벡터 블록입니다.
- V_j: j-번째 Value 벡터 블록입니다.
- d: Key 벡터의 차원입니다. 이를 사용해 스케일링을 적용하여 안정된 Attention 값을 계산합니다.
- A_{ij}: j-번째 블록에 대한 Attention 스코어입니다. 이는 Query 벡터 q_i와 j-번째 Key 벡터 블록 K_j 간의 연산 결과입니다.
- o_i: 최종 Attention 출력입니다. A_{ij}를 j-번째 Value 벡터 V_j와 곱해 합산한 값입니다.

How: vLLM – Method: PagedAttention

PagedAttention



How: vLLM – Method: PagedAttention

PagedAttention

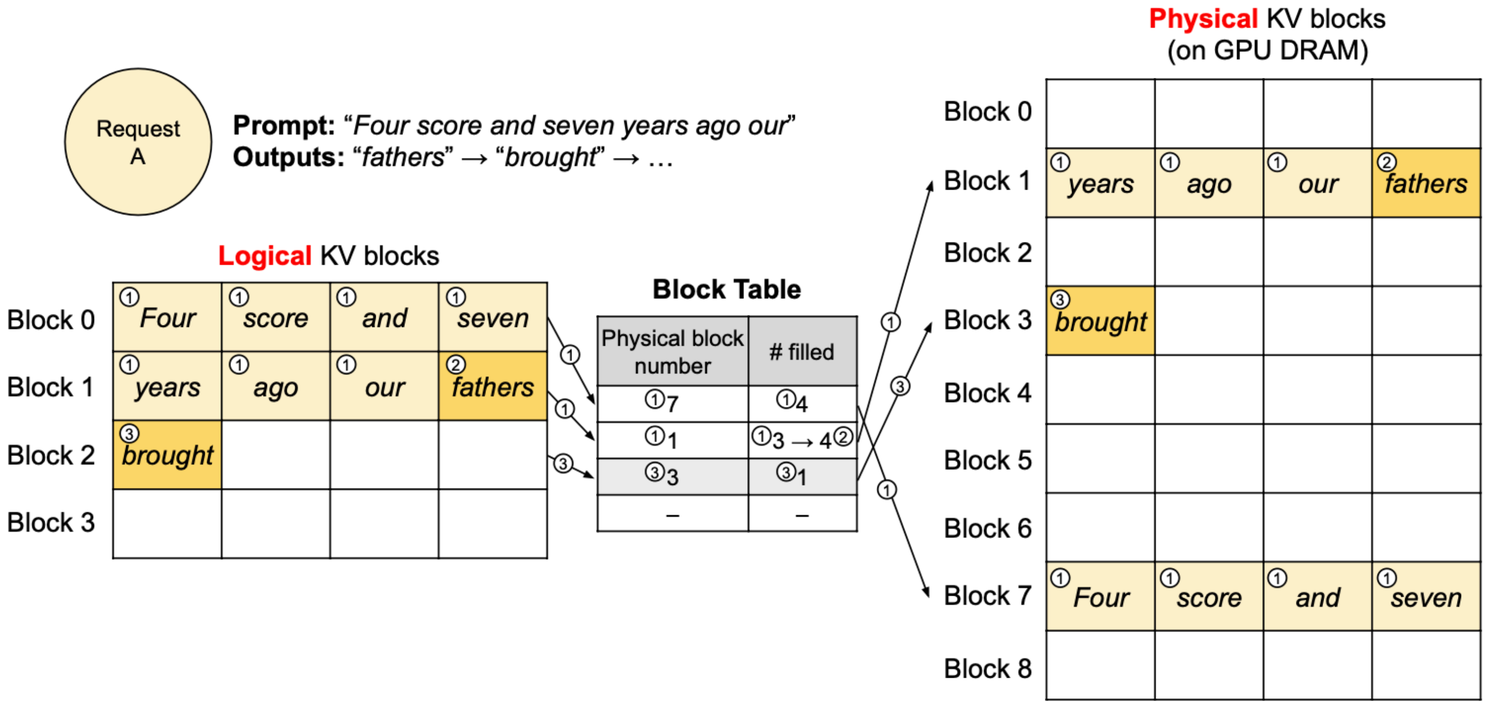


Figure 6. Block table translation in vLLM.

1. 주어진 프롬프트 토큰들을 논리적 KV 블록에 저장, (ex: 논리블록 0, 1에 저장, 물리블록 7, 1에 할당, 블록 테이블에 기록)

2. PagedAttention 알고리즘을 사용하여 새로운 토큰 생성 후 마지막 논리 블록에 저장 후 블록 테이블 업데이트

3. 마지막 논리 블록이 가득 차면, 새로운 물리적 블록 할당 후 새로운 KV Cache를 저장

How: vLLM – Method: PagedAttention

PagedAttention

0. Before generation.

Seq A

Prompt: “Alan Turing is a computer scientist”
Completion: “”

Logical KV cache blocks

Block 0				
Block 1				
Block 2				
Block 3				

Block table

Physical block no.	# Filled slots
–	–
–	–
–	–
–	–

Physical KV cache blocks

Block 0				
Block 1				
Block 2				
Block 3				
Block 4				
Block 5				
Block 6				
Block 7				

How: vLLM – Method: Parallel Sampling

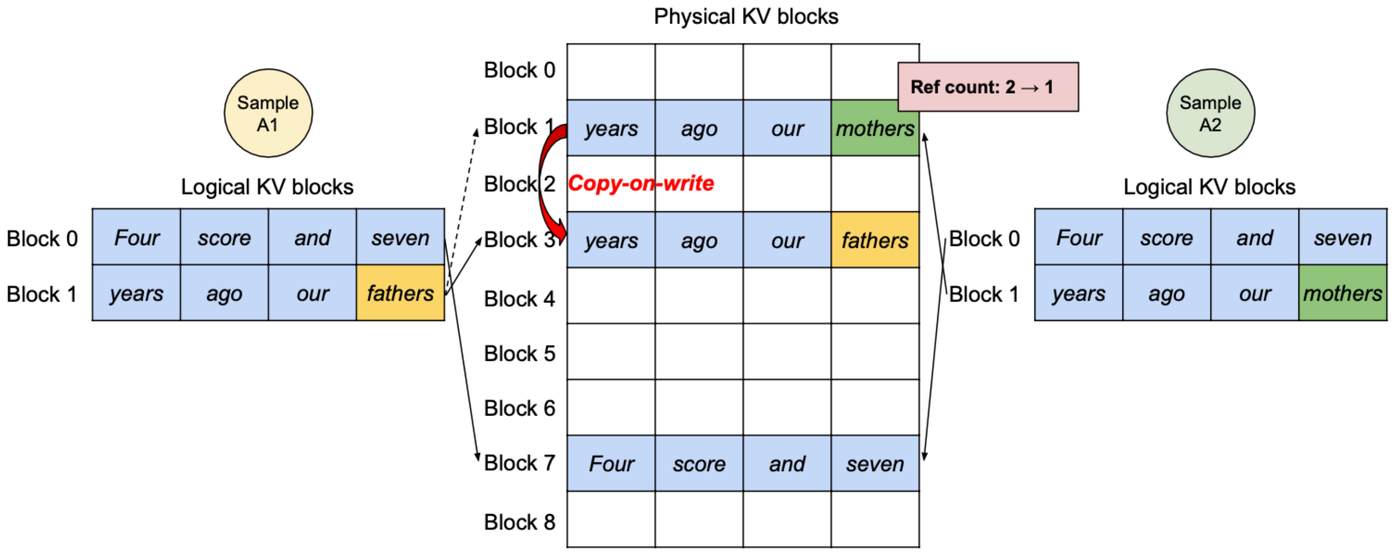


Figure 8. Parallel sampling example.

Key Idea: 중복되는 블록은 같은 블록을 참조하여 메모리를 절약한다.

[동작과정]

1. A1, A2는 동일한 프롬프트를 공유, 각각의 논리 블록 0,1은 물리 블록 7, 1에 참조됨 참조 카운트는 2

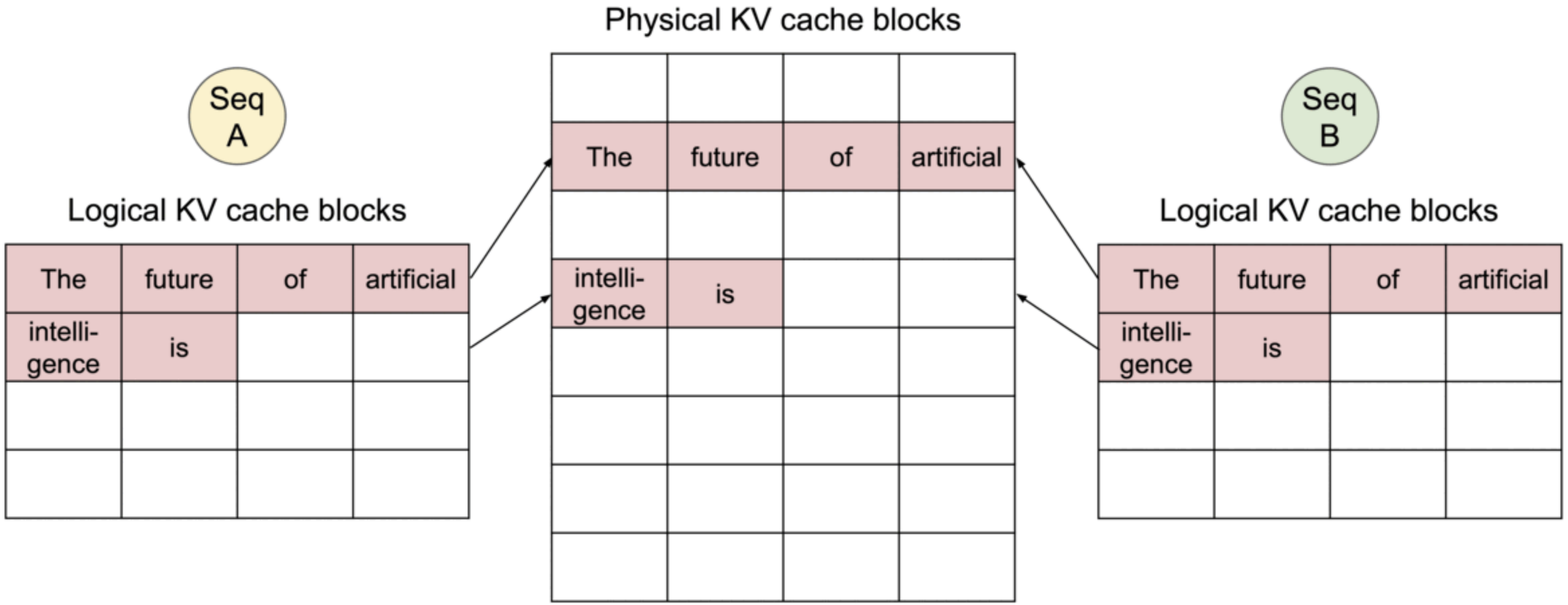
2. A1, A2가 디코딩 과정에서 서로 다른 토큰을 생성함
A1이 마지막 논리 블록에 기록해야 하는 경우, 물리 블록의 참조 카운트가 2인 것을 인식 -> Copy-on-Write 수행

3. Copy-on-Write: 해당 물리 블록의 데이터를 새 물리 블록으로 복사한 후 참조 카운트를 1 줄임,
새로운 블록은 참조 카운트 1
A2는 여전히 기존 물리 블록을 참조

4. 데이터 쓰기: 물리 블록의 참조 카운트가 1이면 데이터를 기록
A2가 토큰을 기록할 때 참조 카운트가 1이므로 그대로 기록

How: vLLM – Method: Parallel Sampling

0. Shared prompt: Map logical blocks to the same physical blocks.



How: vLLM – Method: Beam Search

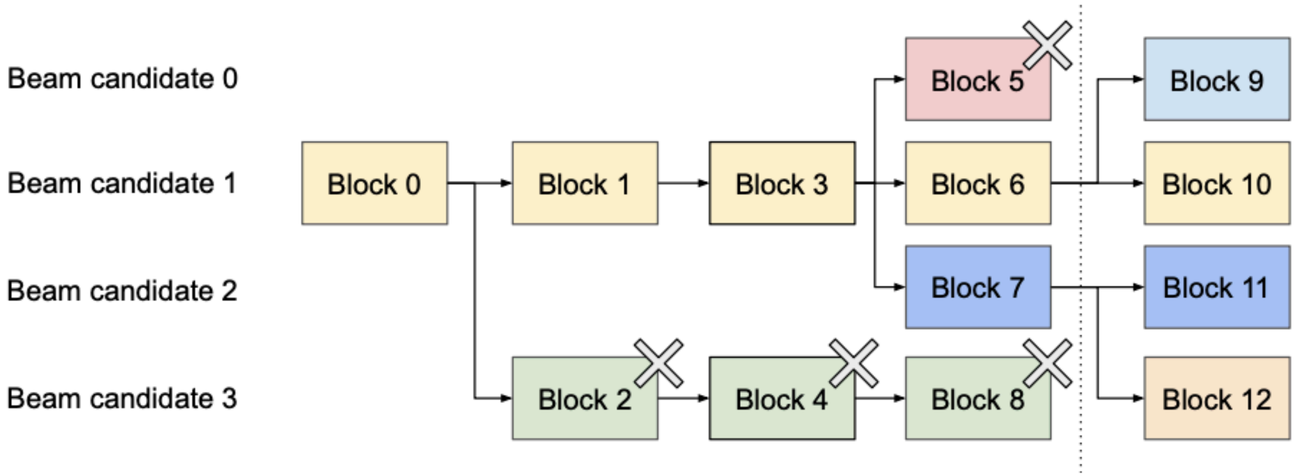


Figure 9. Beam search example.

Key Idea: 중복되는 블록은 같은 블록을 참조하여 메모리를 절약한다.

[동작과정] 그림에서 beam width: k=4

1. “Block0”를 프롬프트로 간주하면 모든 블록은 “Block0”를 공유한다.

2. 앞서 설명한 방식인 Copy-on-Write 기법을 이용하여 Block을 늘려나간다.

3. 블록의 참조카운트가 0이 된 블록은 제거 되었다고 볼 수 있으며, k개의 블록 width 만을 만들 수 있다.

How: vLLM – Metric

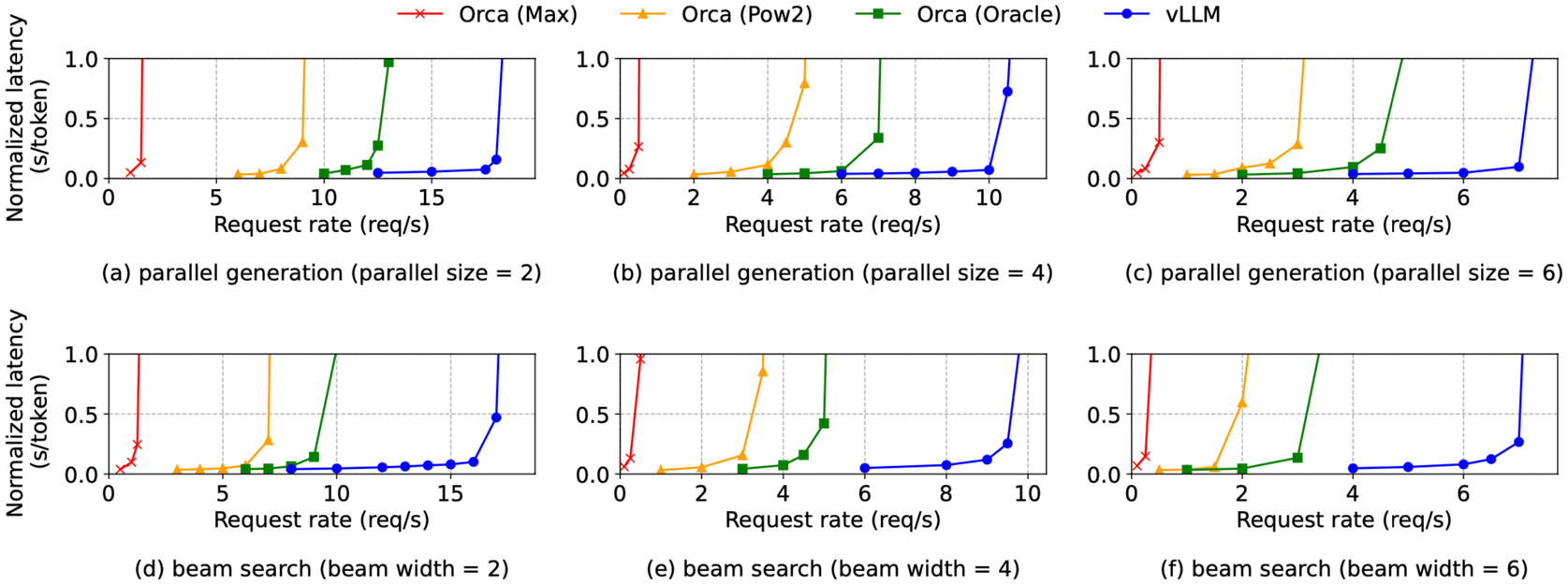
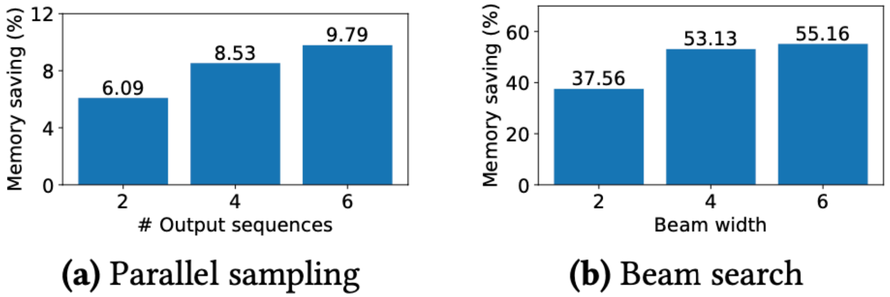


Figure 14. Parallel generation and beam search with OPT-13B on the Alpaca dataset.

초당 요청처리 수 비교



(a) Parallel sampling **(b)** Beam search

Figure 15. Average amount of memory saving from sharing KV blocks, when serving OPT-13B for the Alpaca trace.

메모리 절약 추이

How: vLLM – Scheduling and Preemption

Done? No…

연산 과정에서 GPU 메모리를 초과해서 KV Cache가 생성되면 어떻게 해야 할까?

context: (1) Which blocks should it evict? (2) How to recover evicted blocks if needed again? Typically, eviction policies use heuristics to predict which block will be accessed furthest in the future and evict that block. Since in our case we

1. 어떤 블록을 내보낼지?
2. 내보낸 블록을 필요할 때 어떻게 복구 시킬지?

Eviction Policy : All-or-Nothing
요청이 포함하는 Sequence의 블록을 전체 단위로 해제하거나 그대로 유지
새로운 요청을 중단하고 선점된 요청을 우선적으로 처리.

Recover Policy : Recomputation
해제된 블록이 다시 필요할 때 해당 블록을 처음부터 다시 계산하여 복구

thest in the future and evict that block. Since in our case we know that all blocks of a sequence are accessed together, we implement an all-or-nothing eviction policy, i.e., either evict all or none of the blocks of a sequence. Furthermore, multiple sequences within one request (e.g., beam candidates in one beam search request) are gang-scheduled as a *sequence group*. The sequences within one sequence group are always preempted or rescheduled together due to potential memory sharing across those sequences. To answer the second question of how to recover an evicted block, we consider two techniques:

Swapping. This is the classic technique used by most virtual memory implementations which copy the evicted pages to a swap space on the disk. In our case, we copy evicted blocks to the CPU memory. As shown in Fig. 4, besides the GPU block allocator, vLLM includes a CPU block allocator to manage the physical blocks swapped to CPU RAM. When vLLM exhausts free physical blocks for new tokens, it selects a set of sequences to evict and transfer their KV cache to the CPU. Once it preempts a sequence and evicts its blocks, vLLM stops accepting new requests until all preempted sequences are completed. Once a request completes, its blocks are freed from memory, and the blocks of a preempted sequence are brought back in to continue the processing of that sequence. Note that with this design, the number of blocks swapped to the CPU RAM never exceeds the number of total physical blocks in the GPU RAM, so the swap space on the CPU RAM is bounded by the GPU memory allocated for the KV cache.

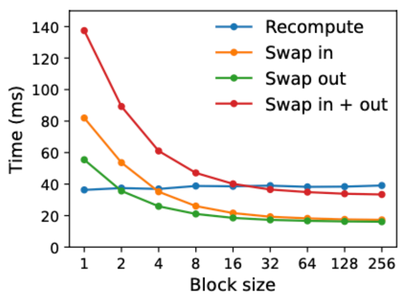
Recomputation. In this case, we simply recompute the KV cache when the preempted sequences are rescheduled. Note that recomputation latency can be significantly lower than the original latency, as the tokens generated at decoding can be concatenated with the original user prompt as a new prompt—their KV cache at all positions can be generated in one prompt phase iteration.

The performances of swapping and recomputation depend on the bandwidth between CPU RAM and GPU memory and the computation power of the GPU. We examine the speeds of swapping and recomputation in §7.3.

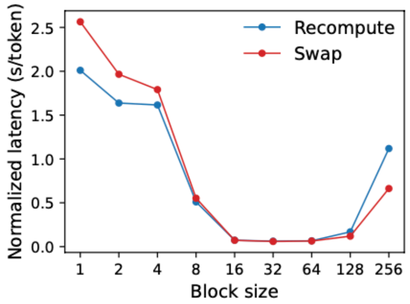
How: vLLM – Block Size

가장 적절한 Block Size는??

The choice of block size can have a substantial impact on the performance of vLLM. If the block size is too small, vLLM may not fully utilize the GPU's parallelism for reading and processing KV cache. If the block size is too large, internal fragmentation increases and the probability of sharing decreases.



(a) Microbenchmark



(b) End-to-end performance

since the sequences become shorter than the block sizes. In practice, we find that the block size 16 is large enough to efficiently utilize the GPU and small enough to avoid significant internal fragmentation in most workloads. Accordingly, vLLM sets its default block size as 16.

Block Size가 vLLM 퍼포먼스에 중요한 영향을 준다.

Block Size가 너무 크면?
- **내부 단편화 증가, 블록 공유 가능성 저하**

Block Size가 너무 작으면?
- **KV Cache 연산 효율 저하 -> Latency 증가**

즉, 적당한 Block Size를 찾아야 함!

- 벤치마킹 도구를 이용하여 각 Block Size 별 퍼포먼스를 분석

그 결과, Block Size는 16이 제일 적당하여 default 값으로 채택

How: vLLM - Metric

다른 LLM Serving 시스템과 성능 비교

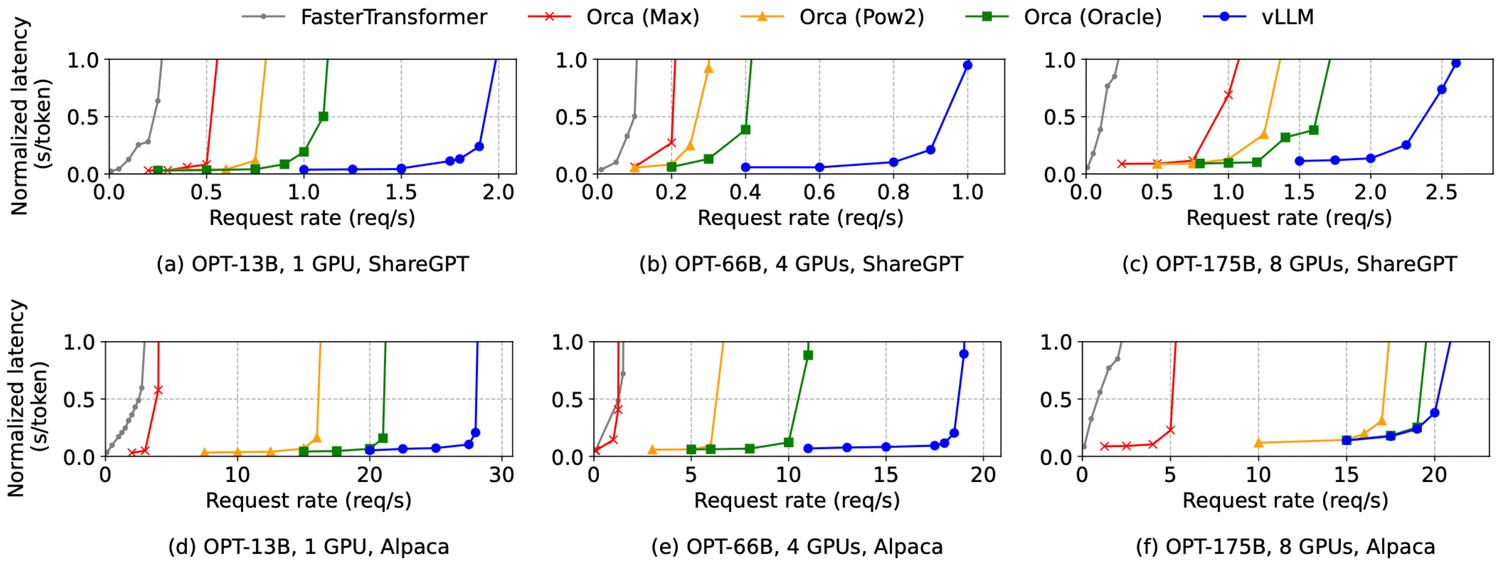


Figure 12. Single sequence generation with OPT models on the ShareGPT and Alpaca dataset

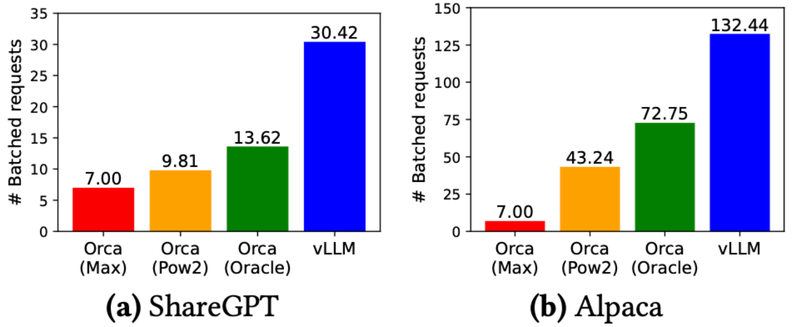


Figure 13. Average number of batched requests when serving OPT-13B for the ShareGPT (2 reqs/s) and Alpaca (30 reqs/s) traces.

- 초당 처리량 비교

- 평균 배치 처리량 비교

일반적으로 vLLM이 더 나은 성능을 보임

How: vLLM – Conclusion

vLLM 논문은…

This paper proposes PagedAttention, a new attention algorithm that allows attention keys and values to be stored in non-contiguous paged memory, and presents vLLM, a high-throughput LLM serving system with efficient memory management enabled by PagedAttention. Inspired by operating systems, we demonstrate how established techniques, such as virtual memory and copy-on-write, can be adapted to efficiently manage KV cache and handle various decoding algorithms in LLM serving. Our experiments show that vLLM achieves 2-4× throughput improvements over the state-of-the-art systems.

[아이디어]
- 운영체제의 가상메모리 알고리즘을 이용하여 문제 해결을 한 아이디어가 대단함

[성능]
- 실제 성능적으로 우수함
- 메모리 단편화를 매우 크게 줄임
- Block 단위 메모리 공유로 Beam Search의 메모리 절약 성능이 놀라움

한계

전체 KV Cache의 크기를 줄인 것은 아님, 메모리 효율을 높인 것임

InfiniGen

InfiniGen: Efficient Generative Inference of
Large Language Models with Dynamic KV Cache Management
2024.06.28

How: InfiniGen – Problem

and repeated computation. Unlike the model weights, however, the KV cache scales with the output sequence length, often consuming even more memory capacity than the model weights. As the demand for longer sequence lengths (along with larger batch sizes) continues to grow, the issue of the KV cache size will become more pronounced in the future.

Meanwhile, modern LLM serving systems support offloading data to the CPU memory to efficiently serve LLMs within the hardware budget [5,57]. These offloading-based inference systems begin to support even offloading the KV cache to the CPU memory, thereby allowing users to generate much longer contexts beyond the GPU memory capacity. However, transferring the massive size of the KV cache from the CPU memory to the GPU becomes a new performance bottleneck in LLM inference.

점점 긴 Sequence 길이를 원한다…

KV Cache 크기가 모델 크기보다 더 커지는 일이 빈번하다…

앞으로 KV Cache 크기 이슈는 더 많이 논의 될 것이다…

엄청난 크기의 KV Cache를 CPU 메모리와 GPU 메모리 사이에서 전달하는 것은 추론과정에서 병목지점이 된다.

How: InfiniGen – Problem

In this work, we propose InfiniGen, a KV cache management framework designed to synergistically work with modern offloading-based inference systems. InfiniGen builds on two key design principles. First, it speculates and chooses the KV cache entries that are critical to produce the next output token, dropping the non-critical ones, by conducting a minimal *rehearsal* of attention computation for Layer i at Layer $i-1$. Second, it leverages the CPU memory capacity and maintains the KV cache pool on the CPU, rather than on the GPU, to ensure that the critical KV cache values can be identified for all outputs and layers with a large *window size* while alleviating the concerns about limited GPU memory capacity for long content generation.

with low data transfer overhead. The key design principle behind InfiniGen is to exploit the abundant CPU memory capacity to increase the *window size* when identifying the important tokens in the KV cache. As such, the majority of the

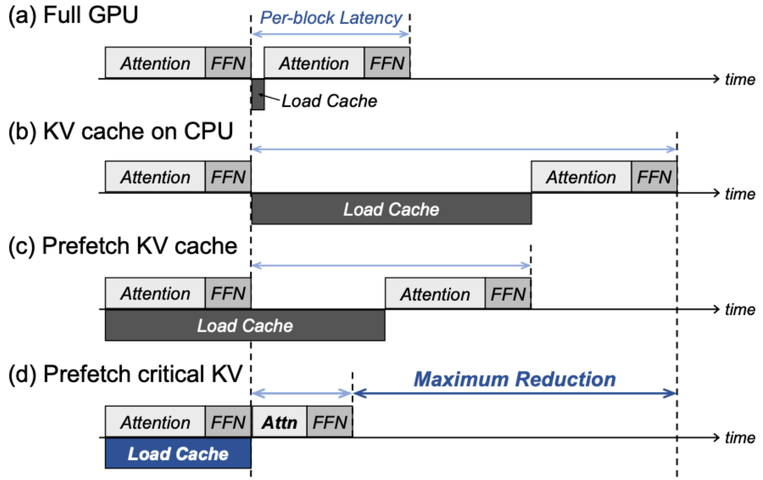


Figure 3: Comparison between different execution styles of Transformer blocks.

InfiniGen은 이 문제를 해결하기 위해 2가지 접근법을 제안합니다.

1. **중요한 KV Cache 만으로 다음 토큰을 계산, 필요없는 토큰은 제거**
2. **KV Cache Pool을 GPU가 아닌 CPU에서 관리**

이를 이용하면 더 큰 Window Size로 더 중요한 KV Cache 토큰을 관리 가능함

How: InfiniGen – Method: Architecture

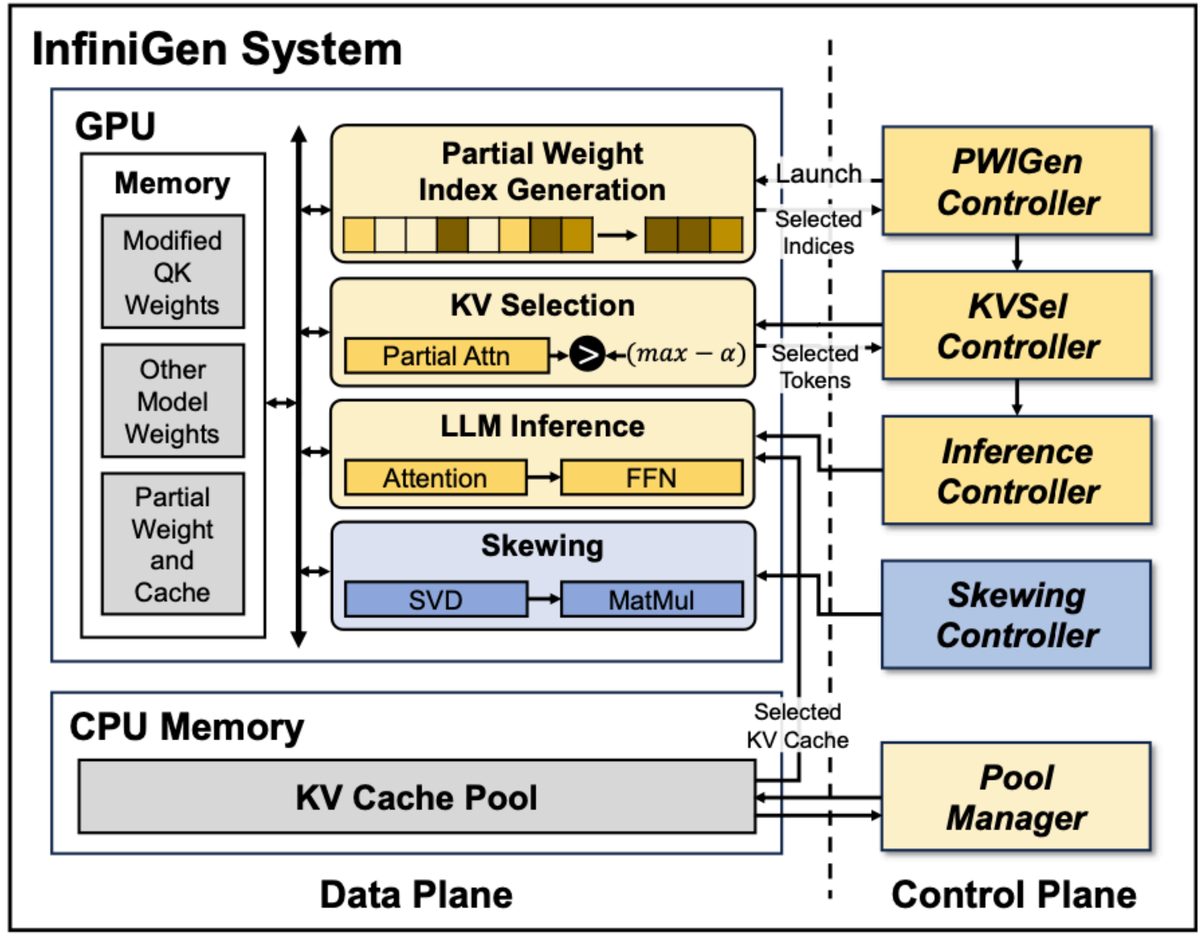


Figure 6: Overview of InfiniGen design.

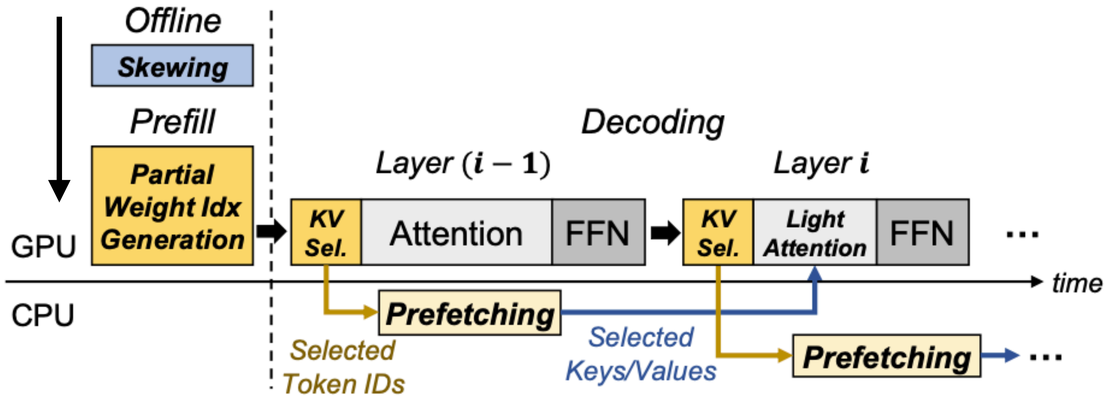


Figure 8: Operation flow of the prefetching module of InfiniGen.

1. Offline 단계에서는 Skewing을 수행
2. Prefill 단계에서는 Partial Weight Idx Generation 수행
3. Decoding 과정에 KV Selection 과정 수행
4. KV Selection으로 선택된 토큰들이 Prefetching으로 다음 레이어 연산에 이용

Skewing, PWIGen, KV Selection 등은 도대체 무엇이고 어떻게 작동하고 어떤 역할을 하는것일까?

How: InfiniGen – Method: Skewing

Key Idea: **Query, Key를 이용한 Attention 연산에서 중요한 영향을 미치는 Weight는 어느 부분일까?**

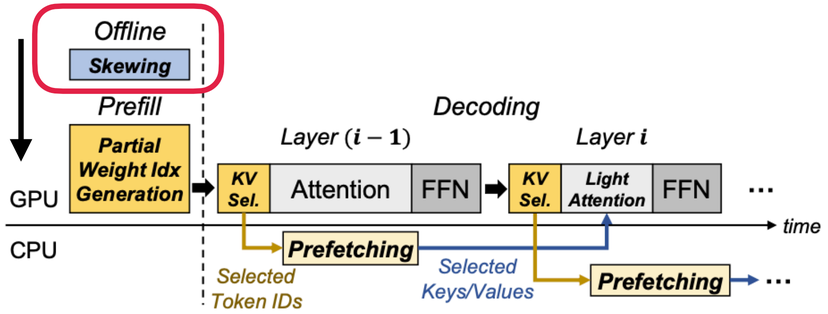
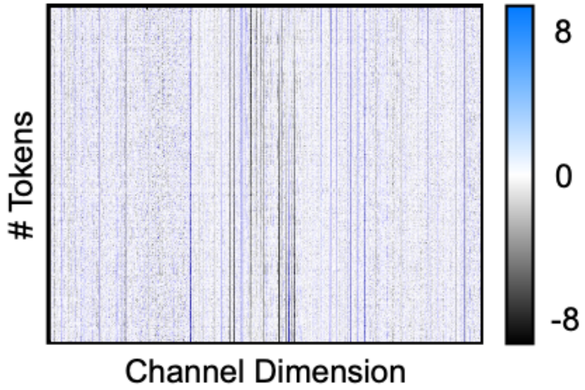


Figure 8: Operation flow of the prefetching module of InfiniGen.



(b) Query

Skewed Partial Weight. We observe that the attention score highly depends on a few columns in the query and key matrices. Figure 7(b) shows the values in a query matrix of Layer 18 of the OPT-13B model, where the column-wise patterns indicate that there exist certain columns with large magnitudes in the matrix; we observe the same patterns in the key and query matrices across different layers and models. The large magnitude columns have a great impact on the attention pattern because the dot product between the query and key is highly affected by these few columns. The column-wise pattern in the attention input indicates that there is little variance between each row in the outlier channels. Thus, the dot prod-

Attention 결과가 Query, Key 행렬의 일부 Column에 의존 한다는 것을 발견

실제 Column-Wise 한 패턴을 보였고 크기가 클 수록 (8, -8 에 가까울 수록) Attention 연산 결과에 큰 영향을 끼친다!

그럼 Query, Key의 중요한 Column 만을 이용하여 계산해도 Attention 결과는 비슷하지 않을까?

그런데 어떻게?
기존 Attention 결과와의 차이는?



How: InfiniGen – Method: Skewing

Key Idea: **SVD를 이용하여 중요한 부분을 더 강조하자!**

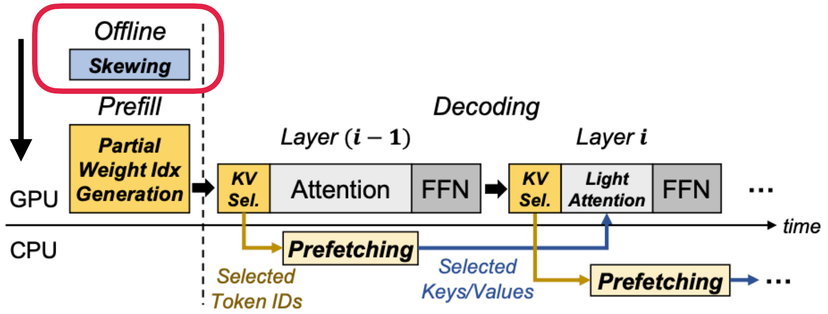


Figure 8: Operation flow of the prefetching module of InfiniGen.

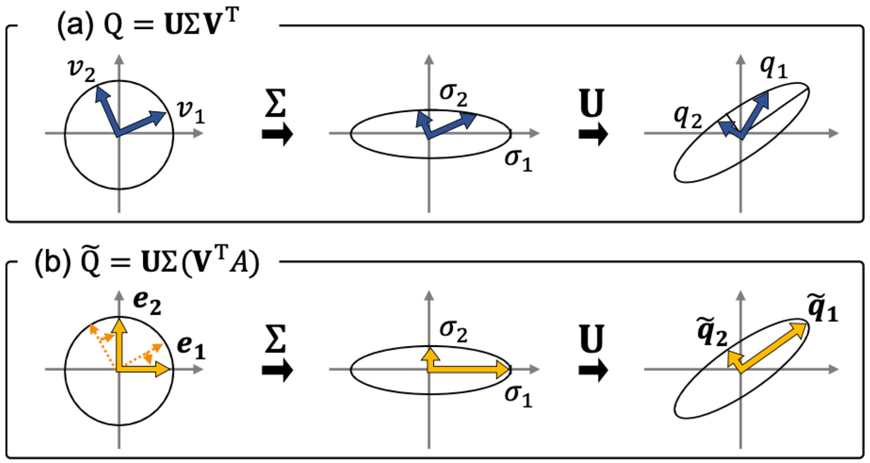


Figure 1: Transformation from matrix $\mathbf{V}^T$ to matrix Q in terms of SVD. The orthogonal matrix A maximizes the difference in magnitude between the column vectors of Q.

SVD: Singular Value Decomposition - 특이값 분해

most. Specifically, we first decompose the query matrix using SVD and obtain $\mathbf{U}$, Σ, and $\mathbf{V}$. We then set A to orthogonal matrix $\mathbf{V}$ to align the column vectors with the standard unit vectors as $\mathbf{V}^T\mathbf{A} = \mathbf{V}^T\mathbf{V} = I$, where I is an identity matrix. We formulate the skewed query matrix as follows:

$$\tilde{Q} = Q \times A = \mathbf{U}\Sigma\mathbf{V}^T \times A = \mathbf{U}\Sigma\mathbf{V}^T \times \mathbf{V} \quad (3)$$

In this way, we can make a few columns with large magnitudes in $\tilde{Q}$ without altering the result of computation, as discussed in Section 2.4.

Query 행렬을 SVD하고 직교 행렬인 A를 행렬곱하면 그림 (b)와 같이 q 벡터간 대비가 더 커짐을 알 수 있다.

이해가 잘 안된다. 더 쉽게 설명이 필요하다.

How: InfiniGen – Method: Skewing

most. Specifically, we first decompose the query matrix using SVD and obtain $\mathbf{U}$, Σ, and $\mathbf{V}$. We then set A to orthogonal matrix $\mathbf{V}$ to align the column vectors with the standard unit vectors as $\mathbf{V}^T\mathbf{A} = \mathbf{V}^T\mathbf{V} = I$, where I is an identity matrix. We formulate the skewed query matrix as follows:

$$\tilde{Q} = Q \times A = \mathbf{U}\Sigma\mathbf{V}^T \times A = \mathbf{U}\Sigma\mathbf{V}^T \times \mathbf{V} \quad (3)$$

In this way, we can make a few columns with large magnitudes in $\tilde{Q}$ without altering the result of computation, as discussed in Section 2.4.

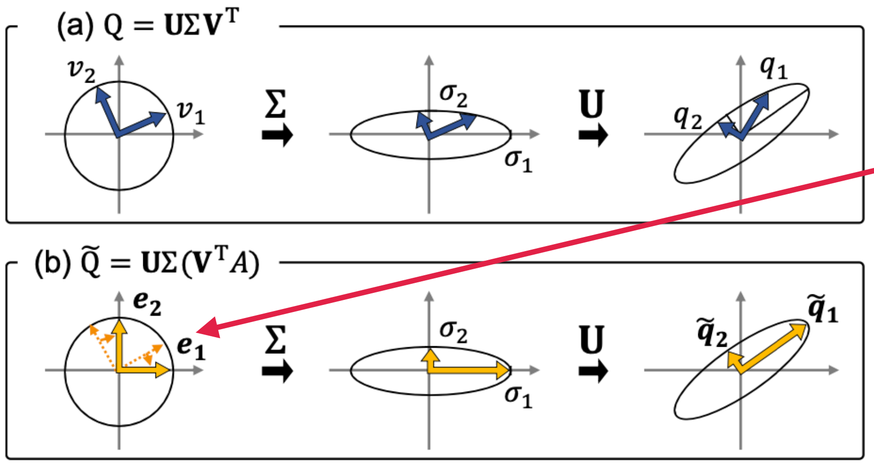


Figure 1: Transformation from matrix $\mathbf{V}^T$ to matrix Q in terms of SVD. The orthogonal matrix A maximizes the difference in magnitude between the column vectors of Q.

A 행렬은 무엇인가? - Skewing을 하기 위해서 만들어진 행렬이다.

Skewing은 무엇인가? - 벡터 간 대비를 키워 중요한 벡터의 영향력을 키운다.

왜 벡터 간 대비를 키우는가? - 더 적고 중요한 벡터 만으로 Attention 값을 계산하기 위해

A를 V 로 맞추는가? - 전치행렬 V는 직교 행렬이다. 즉, VV^T = I 이다.
Identity 행렬로 만들어 진다는 것을 그림의 이 부분으로 맞춘다는 얘기이다.

Ex) 2X2 행렬 I 를 생각해보자.
[1, 0]
[0, 1]
화살표가 나타내는 그림과 일치한다.

결국, SVD를 이용하여 얻은 A 행렬로 중요한 Column을 강조할 수 있다는 것이다.

그렇다면 Skewing 된 행렬을 이용하여 Attention 연산 시 결과는 어떨까?

How: InfiniGen – Method: Skewing

놀랍게도 *Skewed* 된 Query, Key 행렬을 이용한 Attention 결과는 변하지 않는다.

Since the transpose of the orthogonal matrix is the inverse of itself, the proposed operation does not change the final result, as shown in Equation 2 (i.e., this is mathematically equivalent to QK^T, not an approximation):

$$
\begin{aligned}
\tilde{Q} &= X_a \times W_Q \times A, \qquad \tilde{K} = X_a \times W_K \times A \\
\tilde{Q} \times \tilde{K}^T &= X_a \times W_Q \times A \times (X_a \times W_K \times A)^T \\
&= X_a \times W_Q \times A \times A^T \times W_K^T \times X_a^T \\
&= X_a \times W_Q \times W_K^T \times X_a^T \\
&= X_a \times W_Q \times (X_a \times W_K)^T \\
&= Q \times K^T,
\end{aligned}
\tag{2}
$$

where $\tilde{Q}$ and $\tilde{K}$ are skewed query and key matrices, while W_Q and W_K are query and key weight matrices. X_a denotes the attention input. We set the orthogonal matrix A whose direction

단지 Query, Key 행렬의 벡터 간 대비를 키우기만 했을 뿐

그 외에 다른 과정을 수행한 적은 없다.

중요한 벡터만을 이용하여 Attention 결과를 낸 것이 아닌, 그저 Skewed 된 전체 벡터를 이용했기 때문이다.

우리는 행렬 A를 Query Weight, Key Weight 행렬과 곱하면 Skewed 된 가중치 행렬을 각각 얻을 수 있다.

How: InfiniGen – Method: Skewing

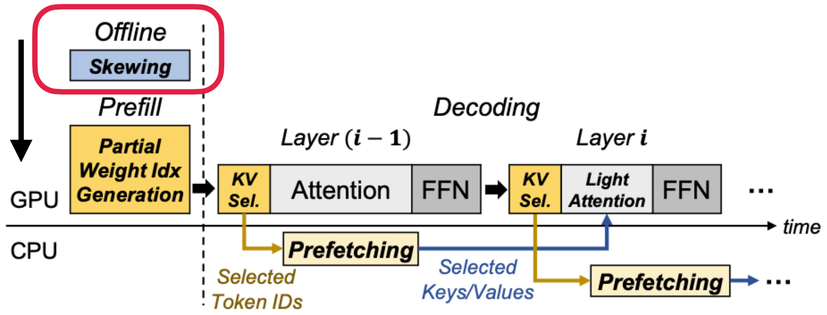


Figure 8: Operation flow of the prefetching module of InfiniGen.

Offline 단계에서

Sample input을 이용하여
각 레이어 별 Query 행렬을 얻을 수 있다.

SVD를 이용하면
각 레이어 별 A 행렬을 얻을 수 있다.

A 행렬을 이용하여
Query Weight, Key Weight 들을 Skewing 할 수 있다.

그리고 Skewing은 한번만 이루어져,
런타임에서는 수행 안하여 오버헤드가 없다고 볼 정도이다.

그러면 우리는 Query, Key 행렬을 얻을 때
좀 더 강조된 결과로 받을 수 있다!

이때, Skewing만 했을 때는 기존 Attention 결과에
영향을 주지 않는 다는 것을 명심해라

Prefetching Scheme. Figure 8 shows the operation flow of the prefetching module in InfiniGen. In the offline phase, InfiniGen modifies the *weight* matrices to generate skewed query and key matrices. To achieve this, InfiniGen first runs the forward pass of the model once with a sample input. During this process, InfiniGen gathers the query matrix from each layer and performs singular value decomposition (SVD) of each query matrix. The skewing matrix (A_i) of each layer is obtained using the decomposed matrices of the query matrix, as shown in Equation 3. This matrix is then multiplied with each of the query and key weight matrices in the corresponding layer. Importantly, after the multiplication, the dimensions of the weight matrices remain unchanged. Note that the skewing is a *one-time* offline process and does not incur any runtime overhead because we modify the weight matrices that are invariant at runtime. As we exploit the column-wise pattern, which stems from the intrinsic property of the model rather than the input, whenever we compute the query and key for different inputs after the skewing, the values exhibit a high degree of skewness, thereby improving the effectiveness of our prefetching module. Note that skewing does not alter the original functionality. Even with the skewing, the attention layer produces identical computation results.

Skewing 과정 = Query Weight, Key Weight 를 변형시켜 좀 더 강조된 Query, Key 행렬을 얻을 수 있게 만듦

How: InfiniGen – Method: Prefill

Key Idea: 강조된 ***Query, Key***의 ***Weight*** 중 일부만을 선택하자

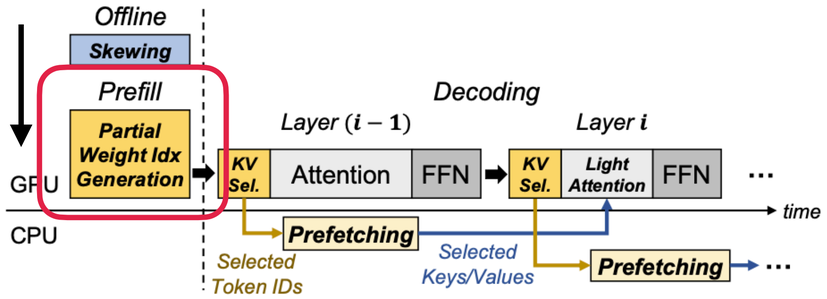


Figure 8: Operation flow of the prefetching module of InfiniGen.

Prefill Stage. In the prefill stage, InfiniGen selects several important columns from the query weight matrix and the key cache to speculate on the attention pattern, and generates *partial* query weight and key cache matrices used in the decoding stage. Figure 9 shows how InfiniGen creates these partial ma-

그런데 중요한 건, 선택한 Column의 인덱스가
Query Weight와 Key Weight 둘 다 같아야 한다.

is essential to select the *same* column indices in the query weight matrix and the key cache to obtain a proper approximation of the attention score. However, the indices of the outlier

그래야 Attention 결과가 적절하게 나온다

어떤 기준으로?
어떻게 Column 인덱스를 선택해야 할까?

How: InfiniGen – Method: Prefill

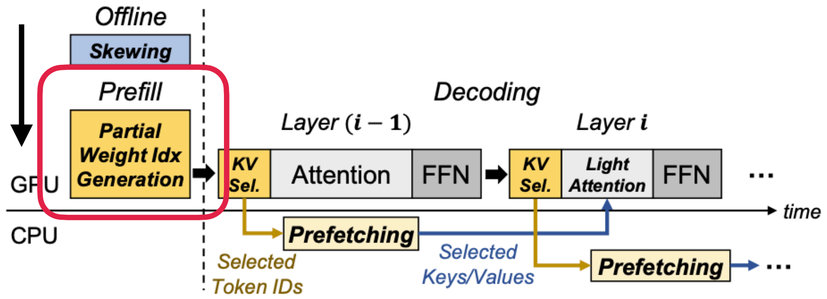


Figure 8: Operation flow of the prefetching module of InfiniGen.

not align exactly. To obtain partial matrices that capture the outliers, we first take the element-wise absolute values of the skewed query and key matrices, then add these two matrices

Query Weight, Key Weight 가 아닌 Query, Key 행렬의 절댓값을 더한다.

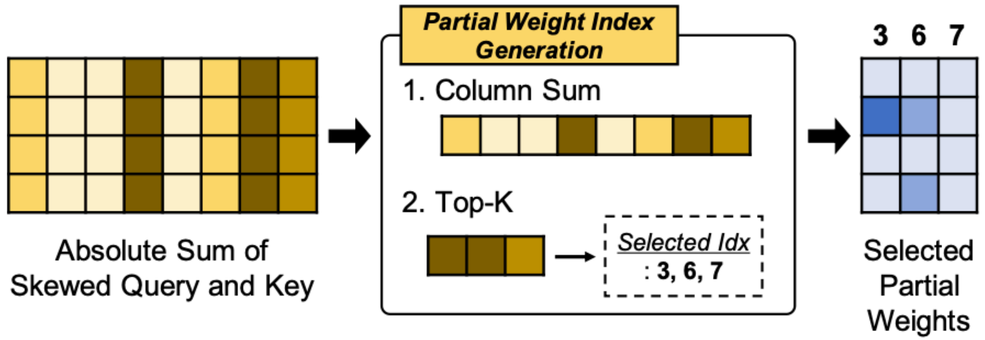


Figure 9: Partial weight generation in the prefill stage.

together. This helps us calculate the sum of each column and perform top-k operation only once while accommodating the outlier columns of both query and key matrices. We then sum the elements in each column and select the top-k columns in the matrix; we choose 30% of the columns in our work. Using

그리고 각 Column별로 합을 구하여 Column Sum을 만든다.

만들어진 Column Sum을 이용하여
Top-K 에 해당하는 Column 인덱스를 구한다.

Prefill 과정 = Skewed Weight의 중요한 Column 인덱스를 구하는 과정

How: InfiniGen – Method: Decoding

Key Idea: 적절한 토큰을 어떻게 선택할까? - Attention 점수 빠르게 예측 후

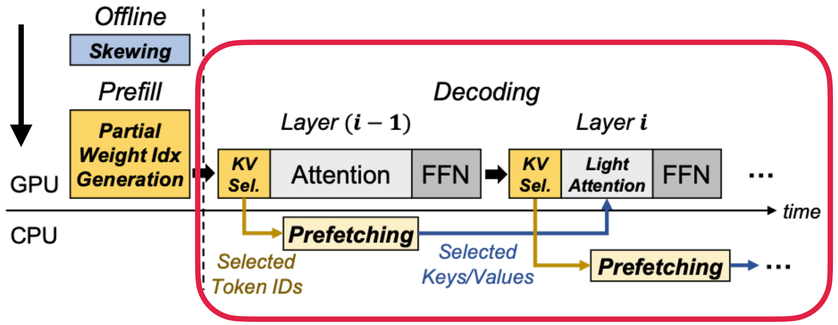


Figure 8: Operation flow of the prefetching module of InfiniGen.

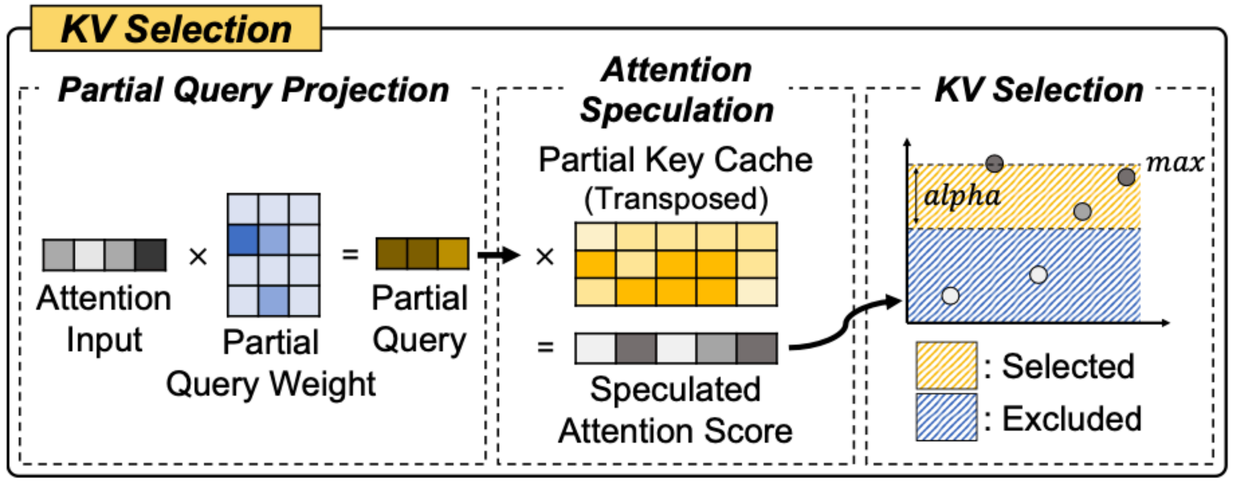


Figure 10: Attention score speculation in the decoding stage.

Decoding Stage. In the decoding stage, InfiniGen speculates on the attention pattern of the next layer and determines the critical keys and values to prefetch. Figure 10 shows how

예측한 Attention 점수로
다음 레이어에 사용 될 KV Cache를 선택해 미리 전달할 수 있다.

InfiniGen computes the *speculated* attention score. At Layer $i-1$, we use the partial query weight matrix and key cache of Layer i, which are identified in the prefill stage, along with the attention input of Layer $i-1$. After multiplying the partial query and partial key cache, InfiniGen selects tokens with high attention scores.

Prefill 단계에서 만들어진 Query와 Key의 Partial Weight들을 이용하면 Attention을 예측할 수 있고

Attention의 Max 값과 Alpha 값을 이용하여
적절한 토큰을 선택하여 다음 레이어에 전달 할 수 있다.

How: InfiniGen – Method: Architecture

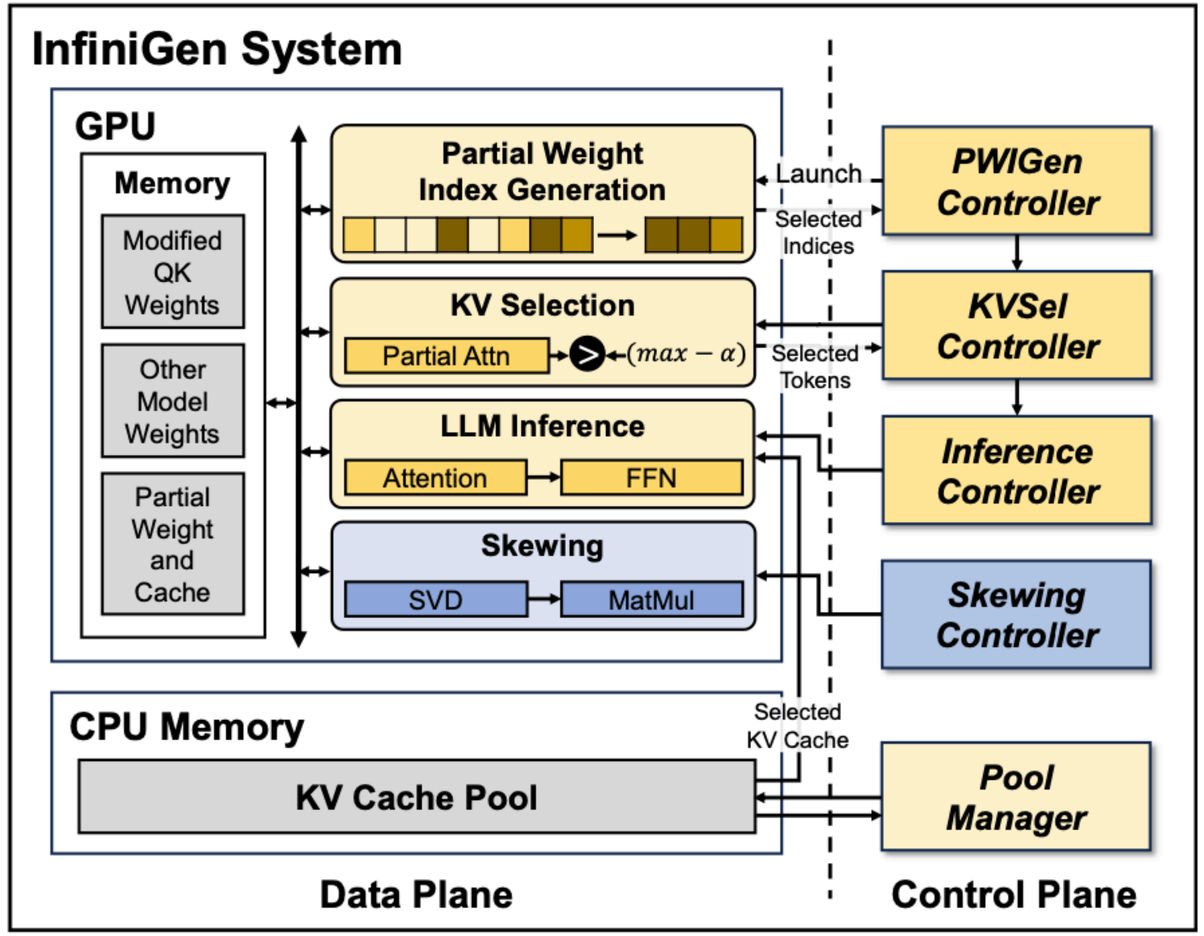


Figure 6: Overview of InfiniGen design.

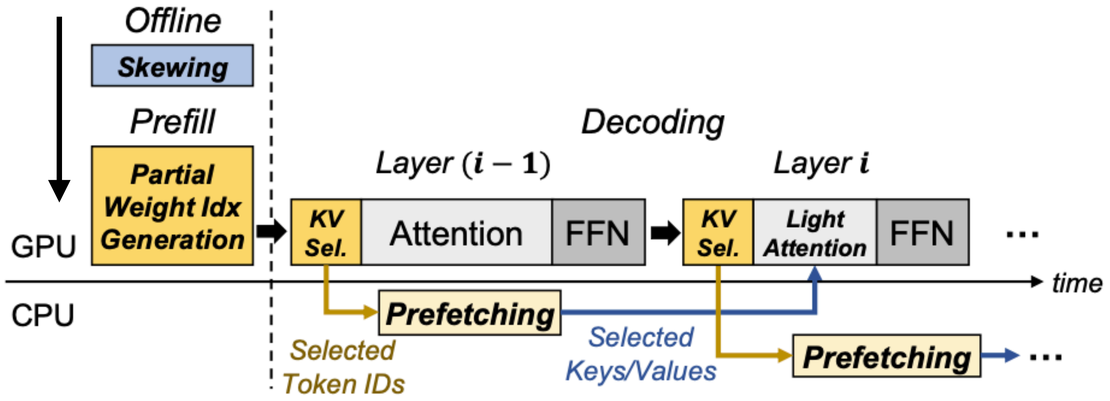


Figure 8: Operation flow of the prefetching module of InfiniGen.

By reducing the amount of KV cache to load and compute, InfiniGen effectively reduces the loading latency (i.e., data transfer from CPU to GPU) while maintaining an output quality similar to that of the original model with a full KV cache. Moreover, as InfiniGen does not require a fixed number of tokens to load from CPU memory, it utilizes only the necessary PCI interconnect bandwidth. InfiniGen initiates speculation

InfiniGen은 모든 KV Cache를 전달하지 않아도 된다!

How: InfiniGen – Method: KV Cache Pool Manager

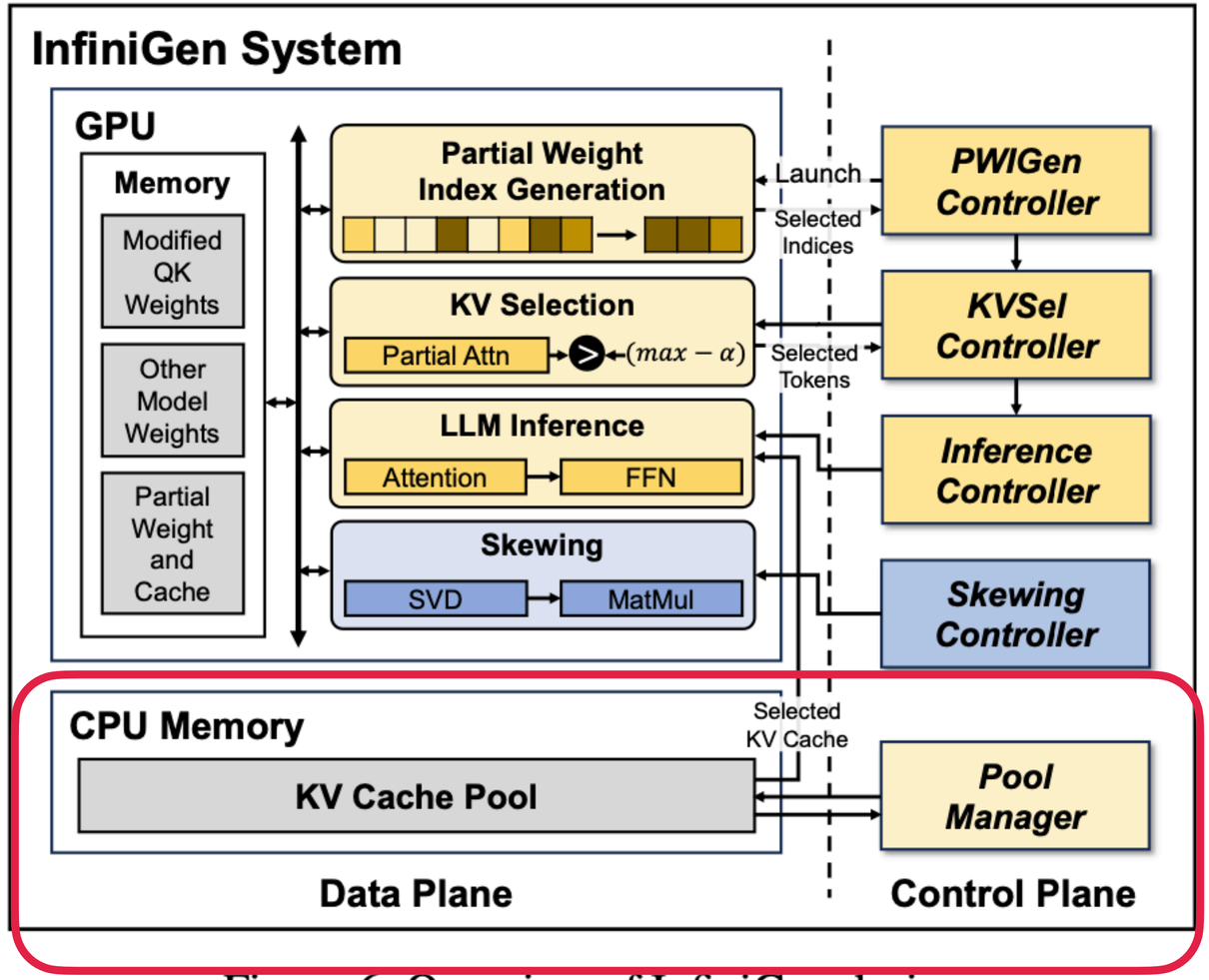


Figure 6: Overview of InfiniGen design.

CPU 메모리를 활용하면 기존보다 더 많은 KV Cache를 저장할 수 있다.

그럼에도 불구하고 CPU 메모리도 한계가 있기 때문에 필요가 없어진 KV Cache는 제거해야한다.

KV Cache Pool Manager는 어떤 정책으로 KV Cache를 제거할까?

How: InfiniGen – Method: KV Cache Pool Manager

FIFO [7, 69, 70] and Least-Recently-Used (LRU) [2]. The FIFO-based policy is easy to implement with low overhead but results in a relatively large accuracy drop since it simply evicts the oldest residing token. The LRU-based policy generally exhibits a smaller decrease in accuracy but often entails a higher runtime overhead. In general, LRU-based policy uses

higher runtime overhead. In general, LRU-based policy uses a doubly linked list with locks to promote accessed objects to

FIFO 전략
- 구현이 쉬움, 낮은 오버헤드, 정확성이 너무 안좋음

LRU 전략
- 정확성이 괜찮음,
- Lock을 써서 런타임 오버헤드가 너무 높음

결론: 둘 다 별로임

KV entries. In the case of the counter-based policy, the pool manager increments a counter for each prefetched KV entry and selects a victim with the smallest count in the KV cache pool. If any counter becomes saturated, all the counter values are reduced by half. We observe that the counter-based policy

Counter-based 전략
1. 사용된 KV Cache는 Count 증가
2. 가장 Count가 적은 KV Cache를 방출
3. 만약 어떤 카운터가 오버플로우 날 것 같다?
   그럼 전체 KV Cache Count를 절반으로 만듬

Counter-based 전략의 성능은?

How: InfiniGen – Method: KV Cache Pool Manager

어떤 KV Cache도 버리지 않았을 때 ->

Counter-based 전략으로 80%의 KV Cache만 사용했을 때 ->

Table 2: Perplexity on WikiText-2 and PTB with 2048 sequence length with or without KV cache memory limits. Lower is better.

Scheme	OPT-6.7B		OPT-13B		OPT-30B		Llama-2-7B		Llama-2-13B	
	Wiki	PTB	Wiki	PTB	Wiki	PTB	Wiki	PTB	Wiki	PTB
100%	11.68	13.86	10.55	12.78	10.14	12.31	5.69	22.53	5.25	31.94
80-FIFO%	19.64	16.82	30.99	33.84	30.66	35.45	22.26	61.88	21.41	32.34
80-LRU%	11.68	13.85	10.55	12.78	10.14	12.31	5.69	22.53	5.25	31.94
80-Counter%	11.68	13.86	10.55	12.78	10.14	12.31	5.69	22.53	5.25	31.94

성능 좋음

which we discuss in Section 5.2. We opt for a counter-based approach due to its simpler design and to avoid atomic memory updates for better parallelism.

구현도 쉽고 Lock도 안써서 오버헤드도 적다.

How: InfiniGen – Metric: Skewing 효과

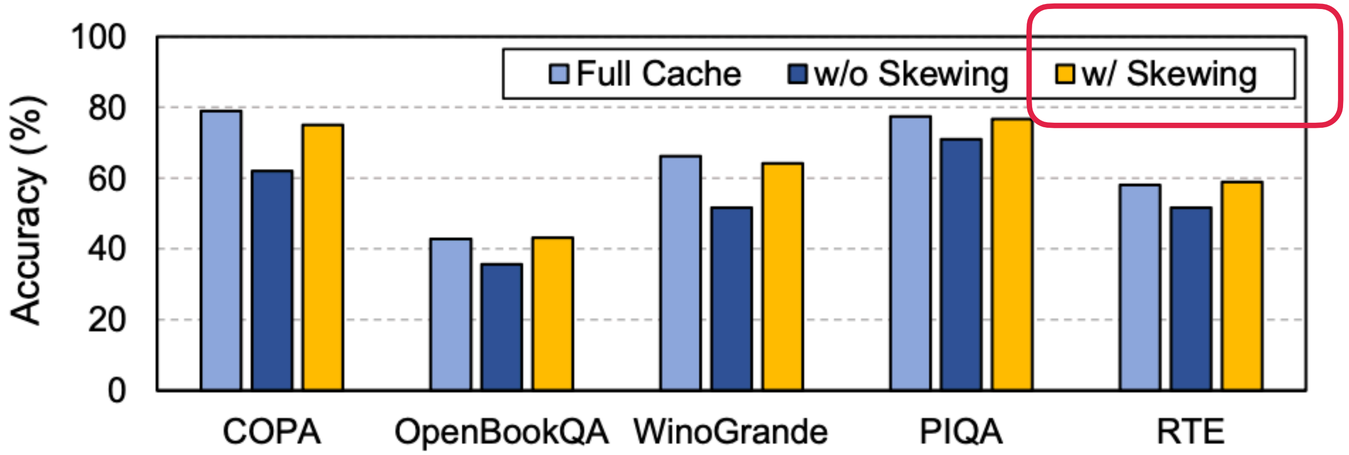


Figure 13: Accuracy on the lm-evaluation-harness benchmark with or without skewing on OPT-6.7B.

trix without skewing. After applying our skewing method, we achieve accuracy similar to the full-cache baseline. Our skew-

Partial Weight 를 만들고 정확성을 비교하면 Skewing을 했을 때가 더 정확성이 올라간다.

How: InfiniGen – Metric: Performance

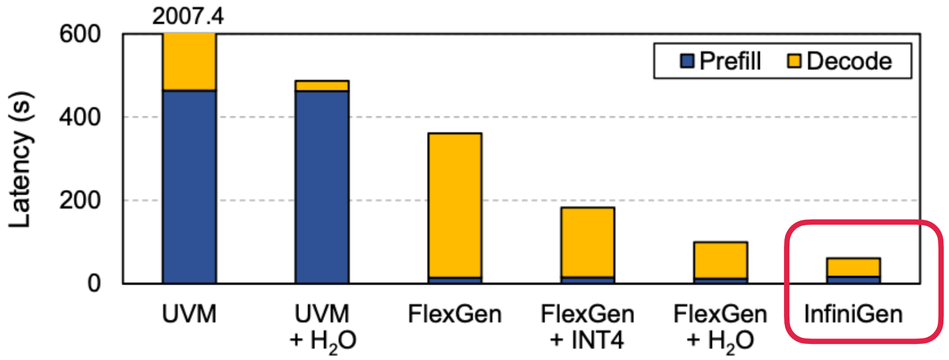


Figure 14: Inference latency on OPT-13B with a sequence length of 2048 (1920 input and 128 output tokens) and a batch size of 20.

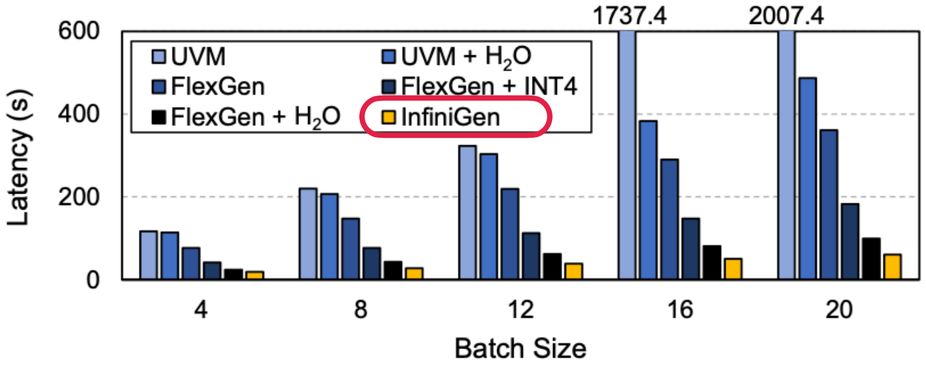


Figure 15: Inference latency for 5 different batch sizes on OPT-13B with a sequence length of 2048 (1920 input and 128 output tokens).

including the prefill and decoding stages. We use the OPT-13B model with 1920 input tokens, 128 output tokens, and a batch size of 20. InfiniGen achieves 1.63×-32.93× speedups over the baselines. The performance benefit mainly comes

다른 시스템과 비교하였을 때,
1.63 ~ 32.93 배 더 나은 성능을 보였음

lower latency than others across the batch sizes (1.28×-34.64×). As the batch size increases, the performance gap between InfiniGen and others becomes larger. UVM and UVM

다른 시스템과 비교하였을 때,
배치 사이즈가 커질 수록 Latency 차이는 더 커짐

How: InfiniGen – Metric: Accuracy

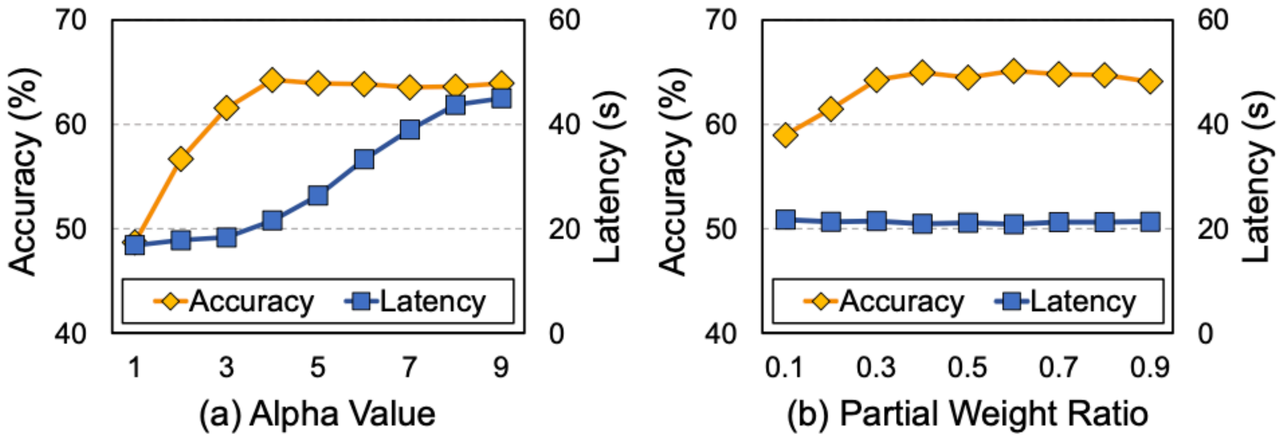


Figure 17: Accuracy and inference latency across (a) alpha values and (b) partial weight ratios.

minus alpha). Increasing alpha results in fetching more KV entries to the GPU, thus increasing inference latency but also improving accuracy. Figure 17(a) shows such trade-offs be-

Alpha 값이 클수록 Latency가 증가하고 Accuracy도 증가한다.
당연한 얘기지만 KV Cache를 더 많이 전달하기 때문이다.

For the alpha values beyond 4, however, since most important tokens are already included, the accuracy does not further increase, while the cost for KV transfers and attention com-

Alpha 값은 4 ~ 5 가 적당한 값인 것으로 확인했다.

the memory consumption overhead). The accuracy also does not noticeably differ beyond a ratio of 0.3. In our work, we opt for a partial weight ratio of 0.3 to achieve better accuracy while considering memory consumption overhead.

Accuracy를 고려했을 때
Partial Weight Ratio는 0.3이 적당했다.

How: InfiniGen – Metric: Overhead

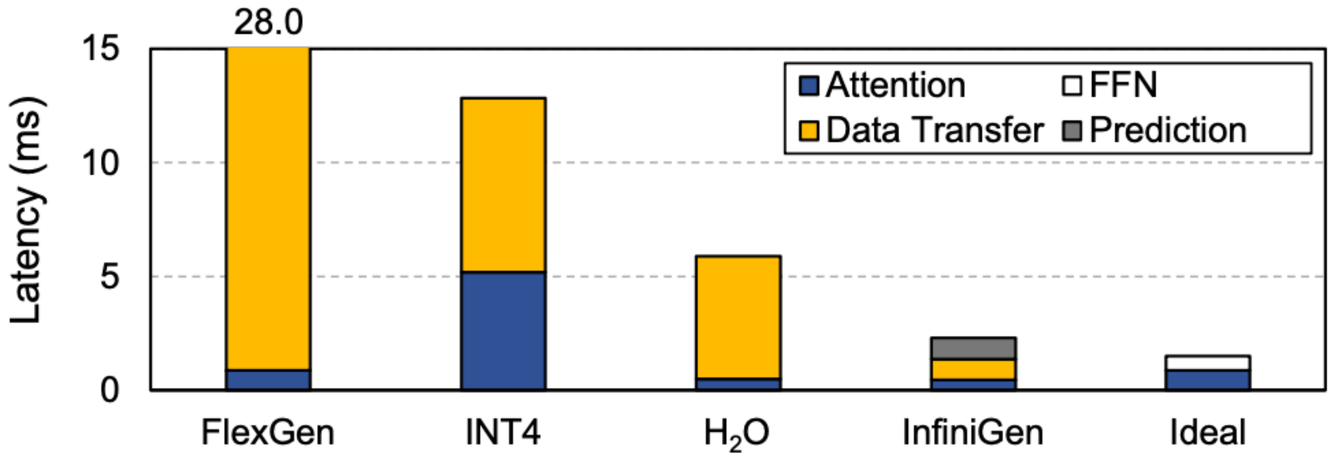


Figure 18: Latency breakdown of a Transformer block for OPT-13B with a sequence length of 2048 and a batch size of 8.

Transformer의 단일 블록에서 시간 소요 측정

KV Cache를 전달하는 과정에서
많은 지연이 발생한다.

ing. Furthermore, InfiniGen is only 1.52× slower than Ideal, while others show 3.90×-18.55× slowdowns.

InfiniGen은 이상치 보다 1.52배 정도만 느리다.

다른건 3.9 ~ 18.55배 더 느리다.

How: InfiniGen – Conculusion

InfiniGen 논문은…

The size of the KV cache poses a scalability issue in high-throughput offloading-based inference systems, even surpassing the model parameter size. Existing KV cache eviction policies show a large accuracy drop and do not efficiently use the interconnect bandwidth when they are employed in offloading-based LLM systems. We propose InfiniGen, an offloading-based dynamic KV cache management framework that efficiently executes inference of large language models. InfiniGen exploits the attention input of the previous layer to speculatively prefetch the KV cache of important tokens. We manipulate the query and key weights to make the speculation more efficient. InfiniGen shows substantially shortened inference latency while preserving language model performance. It also shows much better scalability regarding the batch size, sequence length, and model size compared to prior solutions.

1. **KV Cache를 버릴 때 성능이 감소하는 것을 방지함**
2. **메모리 송수신 대역폭을 고려함**
3. **이전 레이어에서 Attention을 예측해 중요 KV Cache만 전달**

한계

Alpha, Attention Score 예측에 의해 기존 모델과 약간 다른 성능을 보임
KV Cache를 선별하여 일부만 전달해서 어쩔 수 없는 현상은 맞지만 어느정도의 성능 변화가 있는지
좀 더 자세한 검증이 필요해 보임

Q & A

Reference

KV-Cache, Attention 자세히
https://moon-walker.medium.com/long-context%EB%A1%9C-%EC%9D%B8%ED%95%9C-large-kv-cache%EC%9D%98-%EB%AC%B8%EC%A0%9C%EC%A0%90%EA%B3%BC-%ED%95%B4%EA%B2%B0-%EB%B0%A9%EC%95%88-part-i-kv-cache%EC%9D%98-%EB%A9%94%EB%AA%A8%EB%A6%AC-%EC%9A%94%EA%B5%AC%EB%9F%89-025f3d5dea93
https://medium.com/@joaolages/kv-caching-explained-276520203249